



## תיק פרויקט

# WAF – Web Application Firewall

מגישה : הילי אביסרור

ת"ז : 332019132

שם בית הספר : בית חינוך ע"ש הרצוג כפר סבא

שם המנחה : אופיר שביט

תאריך הגשה : 02.05.2026

## תוכן עניינים

|    |                              |
|----|------------------------------|
| 2  | תוכן עניינים                 |
| 4  | מבוא                         |
| 4  | תיאור ראשוני של המערכת       |
| 4  | הגדרת הלקוח                  |
| 4  | הגדרת יעדים ומטרות           |
| 4  | בעיות, תועלות וחשכונות       |
| 5  | סקירת פתרונות קיימים         |
| 5  | סקירת טכנולוגיות הפרויקט     |
| 5  | תיחום הפרויקט                |
| 5  | פירוט תיאור המערכת (אפיון)   |
| 6  | יכולות משתמשים:              |
| 6  | טבלת בדיקות מתוכננות:        |
| 6  | ניהול סיכונים:               |
| 7  | תיאור תחום הידע              |
| 7  | פירוט מצד שרת                |
| 10 | פירוט מצד לקוח               |
| 11 | מבנה / ארכיטקטורה של הפרויקט |
| 11 | תיאור הארכיטקטורה            |
| 12 | תיאור הטכנולוגיות            |
| 13 | תיאור האלגוריתמים המרכזיים   |
| 13 | סביבת הפיתוח                 |
| 14 | פרוטוקול התקשורת             |
| 15 | תיאור מסכי המערכת            |
| 15 |                              |
| 17 | תיאור מבני הנתונים           |
| 18 | סקירת חולשות ואיומים         |
| 19 | מימוש הפרויקט                |
| 19 | מודולים ומחלקות              |
| 22 | קטעי קוד מרכזיים             |
| 25 | מסמך בדיקות                  |
| 26 | מדריך למשתמש                 |
| 26 | עץ קבצי המערכת               |
| 26 | התקנת המערכת                 |
| 27 | אופן שימוש לפי משתמש         |
| 28 | דפלקציה                      |

|    |                                |
|----|--------------------------------|
| 28 | תהליך העבודה .....             |
| 28 | תהליך למידה .....              |
| 28 | תובנות מהתהליך .....           |
| 28 | שיפורים עם משאבים נוספים ..... |
| 29 | ביבליוגרפיה .....              |

## מבוא

### תיאור ראשוני של המערכת

המערכת היא WAF (Web Application Firewall) - מחסום הגנה שמוצב בין הלקוח לשרת ומסנן כל בקשת HTTP לפני שמגיעה ליעדה. הפרויקט כולל שני שרתים: שרת ה-WAF הרץ על פורט 8000, ושרת האפליקציה BookWorm הרץ על פורט 5000. כל בקשה שמגיעה מהדפדפן נקלטת קודם ב-WAF, עוברת סדרת בדיקות אבטחה, ורק לאחר אישור מועברת לשרת BookWorm באמצעות ערוץ מוצפן (AES-GCM על גבי RSA handshake).

שרת BookWorm הוא אתר ספריית ספרים אונליין שנבנה בכוונה עם חולשות - כולל SQL Injection בדף ההתחברות, XSS בדף החיפוש, ו-Path Traversal בדף ההורדה. מטרת השילוב היא ליצור סביבת הדגמה ריאליסטית שבה ניתן לראות בפועל כיצד ה-WAF מונע תקיפות.

בחרתי לפתח מערכת WAF מכיוון שאני מתעניינת בתחום אבטחת המידע ורציתי להבין לעומק כיצד מתקפות רשת מתבצעות ואיך ניתן לזהות ולמנוע אותן. הפרויקט משלב תכנות Python, תקשורת רשת TCP/HTTP, עבודה עם SQLite, הצפנה (RSA + AES-GCM) וממשק ניהול ווב.

### אתגרים עיקריים שנצפו:

- כתיבת Regex מדויק שיזהה מתקפות אמיתיות (כולל double URL encoding) מבלי לחסום בקשות תקינות.
- ניהול תקשורת מקבילית (threading) בין לקוחות מרובים תוך שמירה על ביצועים.
- מימוש (RSA + AES) TLS-like handshake בין ה-WAF לשרת הפנימי.
- שמירה על keep-alive עם timeout של 30 שניות לכל חיבור.

### הגדרת הלקוח

המערכת מיועדת למפתחים ומנהלי מערכות בצוותים קטנים-בינוניים הזקוקים לשכבת הגנה נוספת מבלי לרכוש פתרונות מסחריים יקרים. בנוסף, המערכת מתאימה לסטודנטים ולחוקרים בתחום אבטחת המידע - השרת BookWorm הפגיע בכוונה, ניתן ללמוד על מתקפות ועל הגנה מפניהן בסביבה בטוחה ומבוקרת.

### הגדרת יעדים ומטרות

- זיהוי וחסימה של XSS, SQL Injection, ו-Path Traversal בזמן אמת.
- הגנה מפני Rate Limiting, DDoS, IP Blacklisting ובקשות שבוורות.
- הצפנה מקצה לקצה בין ה-WAF לשרת הפנימי (RSA + AES-GCM).
- תיעוד כל אירוע אבטחה ב-waf\_logs.db עם IP, URL, timestamp וסיבה.
- ממשק Dashboard בזמן אמת עם אפשרות לשינוי מצב Dry Run / Blocking.
- שימוש כסביבת לימוד לתקיפות ואבטחת רשת.

### בעיות, תועלות וחסכונות

#### הבעיה:

אתרים רבים אינם מוגנים מפני מתקפות נפוצות כמו SQLi ו-XSS, ופתרונות מסחריים (Cloudflare WAF, AWS WAF, Imperva) יקרים ומורכבים לצוותים קטנים.

## התועלת:

- חסימה אוטומטית של בקשות זדוניות לפני שמגיעות לשרת.
- תיעוד מלא לניתוח תקיפות ואיתור דפוסים חוזרים.
- מצב Dry Run לניטור ללא חסימה - מתאים לסביבות ייצור שמצריכות בדיקה לפני פעילות ריאלית.
- ערוץ מוצפן (AES-GCM) בין ה-WAF לשרת הפנימי מונע האזנה פנימית.

## סקירת פתרונות קיימים

| פתרון          | יתרונות               | חסרונות לעומת הפרויקט          |
|----------------|-----------------------|--------------------------------|
| Cloudflare WAF | הגנה מתקדמת           | תלות בשירות חיצוני, עלות גבוהה |
| Imperva        | כלי מתקדם             | יקר מאוד                       |
| הפרויקט הנוכחי | קוד פתוח, ניתן ללמידה | הגנה בסיסית                    |

## סקירת טכנולוגיות הפרויקט

המערכת בנויה ב-Python 3 תוך שימוש בפרוטוקולים סטנדרטיים - TCP/HTTP ו-SQLite - ובספריית cryptography לצורכי הצפנה. אין תלות ב-framework חיצוני לשרת ה-WAF עצמו; הוא בנוי על sockets גולמיים לצורך שליטה מלאה על זרימת הנתונים. שרת BookWorm נבנה גם הוא על sockets עם HTTP parser ידני.

## תיחום הפרויקט

### בתחום הפרויקט:

- סינון וחסימת בקשות HTTP חשודות על שכבת האפליקציה.
- הצפנת תעבורה פנימית בין WAF לשרת BookWorm.
- Dashboard ניהולי לצפייה בלוגים ולשינוי הגדרות.
- סימולציית תקיפות בסביבה מבוקרת (attack\_\*.py).

### לא בתחום הפרויקט:

- הגנה ברמת ה-OS, ניהול הרשאות מערכת הפעלה, עדכון גרסאות.
- גיבוי ושחזור, ניהול משתמשים מרובים, תשלומים.

## פירוט תיאור המערכת (אפיון)

המערכת פועלת כ-Reverse Proxy. כל בקשת HTTP שמגיעה לפורט 8000 נקלטת ב-WAF, עוברת ניתוח, ורק לאחר אישור מועברת לשרת BookWorm בפורט 5000 דרך ערוץ מוצפן.

ה-WAF תומך בשני מצבי פעולה:

Dry Run (ברירת מחדל, DRY\_RUN=True בקוד) - מזהה ומתעד אך לא חוסם, מוסיף Header מיוחד X-WAF-Detected

Blocking - חוסם בפועל ומחזיר קודי שגיאה מתאימים.

## יכולות משתמשים:

יכולות מנהל:  
 הוספה / מחיקה / עריכה של כללים  
 הפעלה וכיבוי של כלל  
 צפייה וסינון של לוגים  
 הגדרת מצב Block / Monitor  
 ניהול Blacklist

יכולות User (מבקר באתר):  
 גישה לאתר המוגן  
 קבלת חסימה במידה והבקשה מפרה כלל  
 קבלת הודעת שגיאה בעת חסימה

## טבלת בדיקות מתוכננות:

| מס' | שם הבדיקה      | קלט                           | פלט צפוי             | כלי הבדיקה          |
|-----|----------------|-------------------------------|----------------------|---------------------|
| 1   | בקשה תקינה     | / GET                         | ALLOW 200            | דפדפן               |
| 2   | SQL Injection  | -- username=' OR 1=1          | BLOCK 403            | דפדפן → login/      |
| 3   | XSS            | <script>alert('xss')</script> | BLOCK 403            | דפדפן → search/     |
| 4   | Path Traversal | file=../../etc/passwd?        | BLOCK 403            | דפדפן → download/   |
| 5   | Rate Limiting  | 30 בקשות מהירות               | 429 אחרי 20          | attack_ratelimit.py |
| 6   | DDoS Detection | 10 IPs שונים בשנייה           | Challenge<br>?=3+7   | attack_ddos.py      |
| 7   | IP Blacklist   | 3 התקפות מ-IP זהה             | 403 לכל בקשה<br>הבאה | attack_blacklist.py |
| 8   | Header גדול    | Header 9KB                    | 431                  | attack_sizelimit.py |
| 9   | Body גדול      | Body 2.1MB                    | 413                  | attack_sizelimit.py |
| 10  | Malformed HTTP | Method לא חוקי, חסר Host      | 400                  | attack_malformed.py |

## ניהול סיכונים:

| סיכון           | תיאור                         | פתרון מתוכנן                 | בוצע בפועל                     |
|-----------------|-------------------------------|------------------------------|--------------------------------|
| False Positive  | חסימת בקשות תקינות            | כיוונון Regex, בדיקות מקיפות | decode ב־double, מילה, decode  |
| ביצועים         | עיכוב בזמן תגובה              | מדידות תקופתיות              | threading לכל חיבור            |
| Encoding Bypass | תוקף מסתיר payload ב-encoding | double URL decode            | urllib.parse.unquote_plus כפול |
| קריסה בעומס     | DDoS אמיתי מוריד שרת          | rate limit + DDoS detection  | deque עם חלון 1 שנייה          |

## תיאור תחום הידע

### פירוט מצד שרת

1. שם היכולת: קליטה וניתוח בקשות HTTP  
**מהות:**

שרת ה-WAF מאזין על פורט 8000 דרך socket TCP. לכל חיבור נפתח thread נפרד (threading.Thread). הפונקציה receive\_full\_request קולטת את הבקשה לאחר קליטה מלאה, הבקשה מועברת למחלקה HTTPRequest ב-http\_parser.py לפירוק לשדות: method, path, version, headers, query\_params, form, json, body.

#### אוסף יכולות:

- פתיחת socket TCP ו-accept לחיבורים נכנסים.
- קליטת נתונים גולמיים תוך בדיקת מגבלות גודל בזמן אמת.
- פירוק request line ל-method, path, version.
- פירוק headers ל-dict; פירוק query string ו-form body.
- זיהוי Content-Type לפירוק (x-www-form-urlencoded / JSON) body.

#### אובייקטים נחוצים:

socket TCP, threading.Thread, HTTPRequest (http\_parser.py), deque, DDoS history.

2. שם היכולת: זיהוי התקפות - SQLi / XSS / Path Traversal  
**מהות:**

הפונקציה is\_request\_suspicious בודקת את כל שדות הקלט: path, query\_params, form, body. כל ערך עובר נרמול כפול (double URL decode + lowercase) לפני הבדיקה. הבדיקה מתבצעת כנגד 3 קבוצות Regex:

- --,SQLI\_PATTERNS: \bor\b\s+1\s\*=\s\*1\b, union\s+select
- XSS\_PATTERNS: <\s\*script\b, on\w+\s\*=, javascript\s\*:, <\s\*img\b
- .encoded %2e%2e%2f וגרסאות \.\. ,/.\.: PATH\_TRAVERSAL\_PATTERNS

#### אוסף יכולות:

- double URL decode: urllib.parse.unquote\_plus
- lowercase normalization למניעת עקיפה ב-casing.
- re.search על כל pattern בקבוצות.
- קריאה ל-record\_suspicious\_activity לעדכון מונה ה-IP.

#### אובייקטים נחוצים:

re module, urllib.parse, blacklist\_map, SQLI\_PATTERNS, XSS\_PATTERNS, PATH\_TRAVERSAL\_PATTERNS.

## 3. שם היכולת: Rate Limiting

## מהות:

הפונקציה detect\_rate\_limiting מגבילה כל IP ל-20 בקשות בחלון של 10 שניות.

ה-state נשמר ב-rate\_map = {ip: [win\_start, count]}. כאשר החלון מסתיים מאפסים את המונה. חריגה מחזירה True ו-WAF שולח 429 Too Many Requests.

## אוסף יכולות:

- ניהול חלון זמן גולש (sliding window) עם time.time().
- thread-safe access דרך rate\_lock (threading.Lock).
- החזרת 429 עם Connection: keep-alive (לא סוגר את החיבור).

## אובייקטים נחוצים:

rate\_map dict, rate\_lock, rate\_max=20, rate\_sec=10

## 4. שם היכולת: DDoS Detection ו-Challenge

## מהות:

המערכת מנהלת deque של (timestamp, ip) עבור כל חיבור.

בכל חיבור חדש מסירים ערכים ישנים מעל שנייה ומספרים IPs ייחודיים. אם יש יותר מ-5 IPs שונים בשנייה נכנסים ל"מצב התקפה" ל-30 שניות.

בזמן ההתקפה כל IP לא מאומת מקבל עמוד IP. (3+7=?). challenge שפותר נכון מתווסף ל-verified\_ips ל-5 דקות.

## אוסף יכולות:

- עדכון conn\_history ב-update\_attack\_state לכל חיבור.
- set() של IPs ייחודיים בחלון השנייה האחרונה.
- שליחת CHALLENGE\_HTML עם form ל-waf\_challenge.
- אחסון verified\_ips עם expiry timestamp.

## אובייקטים נחוצים:

conn\_history (deque), conn\_lock, attack\_until, verified\_ips dict, verified\_lock

## 5. שם היכולת: IP Blacklisting

## מהות:

בכל גילוי payload חשוד קוראים ל-record\_suspicious\_activity(ip).

הפונקציה מנהלת blacklist\_map = {ip: [count, ban\_until]}. אחרי 3 עבירות (MAX\_OFFENSES=3) ה-IP נחסם ל-5 דקות (BLACKLIST\_TIME\_SEC=300). ה-Dashboard מאפשר הוספה/הסרה ידנית דרך waf\_control API/.

## אוסף יכולות:

- is\_ip\_blacklisted בדיקה ראשונה לפני כל עיבוד.
- record\_suspicious\_activity עם lock לבטיחות thread.
- אוטו-reset כאשר ban\_until עבר.
- API להוספה והסרה ידנית דרך blacklist\_lock.

## אובייקטים נחוצים:

blacklist\_map, blacklist\_lock, MAX\_OFFENSES=3, BLACKLIST\_TIME\_SEC=300

## 6. שם היכולת: הצפנה - RSA + AES-GCM מהות:

כל תקשורת בין ה-WAF לשרת BookWorm מוצפנת.

ה-WAF מבצע client handshake : מקבל את המפתח הציבורי RSA מהשרת, מייצר מפתח AES-256 אקראי, מצפין אותו עם RSA-OAEP-SHA256, ושולח לשרת. מכאן ואילך כל הודעה מוצפנת.

### אוסף יכולות:

- generate\_rsa\_keypair : יצירת זוג מפתחות RSA-2048.
- rsa\_encrypt / rsa\_decrypt עם OAEP-SHA256.
- aes\_encrypt: IV אקראי 12 bytes + GCM = IV+tag+ciphertext
- aes\_decrypt : פיצול ciphertext, tag, IV + אימות integrity
- perform\_client\_handshake / perform\_server\_handshake

### אובייקטים נחוצים:

tcp\_by\_size, cryptography library (rsa, padding, hashes, AES, GCM), שליחה/קבלה עם גודל.

## 7. שם היכולת: ניהול מסד נתונים ולוגים מהות:

כל אירוע אבטחה שמוזחה נרשם ל-waf\_logs.db (SQLite) דרך log\_attack(ip, url, reason).

טבלה attack\_logs כוללת id, timestamp, ip, url, reason. ה-Dashboard שולף אירועים דרך waf\_events/api?since\_id=X/ עם LIMIT 100 לתצוגה ב-real-time.

### אוסף יכולות:

- init\_db : יצירת טבלת attack\_logs אם לא קיימת.
- log\_attack: INSERT עם parameterized query
- get\_events: SELECT WHERE id > since\_id ORDER BY id DESC

### אובייקטים נחוצים:

sqlite3, DB\_NAME='waf\_logs.db', attack\_logs table

## פירוט מצד לקוח

1. שם היכולת: ממשק משתמש גרפי - Dashboard  
מהות:

קובץ waf\_proxy\_dashboard.html הוא ה-Dashboard של ה-WAF. הוא נטען כאשר גולשים ל-waf\_events/. הממשק עושה polling כל 2 שניות לשני Endpoints: /waf\_events/api?since\_id=X לאירועים חדשים, ו-waf\_control?action=get\_status/ לסטטוס. הוא מציג טבלת אירועים, מצב WAF (Dry Run / Blocking), ורשימת IPs חסומים.

## אוסף יכולות:

- pollEvents: fetch + insert שורות חדשות לטבלה עם אנימצית flash.
- pollStatus: עדכון mode label ורשימת blacklist.
- toggleMode: POST ל-waf\_control?action=toggle\_mode/.
- addBlacklistIP / removeBlacklistIP: ניהול חסימות ידניות.
- classifyReason: קביעת badge צבעוני לפי סוג ההתקפה.

## אובייקטים נחוצים:

JavaScript fetch API, setInterval (2000ms), DOM manipulation, lastEventId

2. שם היכולת: סקריפטי סימולציית תקיפות  
מהות:

5 סקריפטי Python שמדמים תקיפות ריאליות לצורך בדיקת ה-WAF. כולם עובדים ישירות עם socket TCP גולמי (ללא requests library) לשליחת HTTP raw.

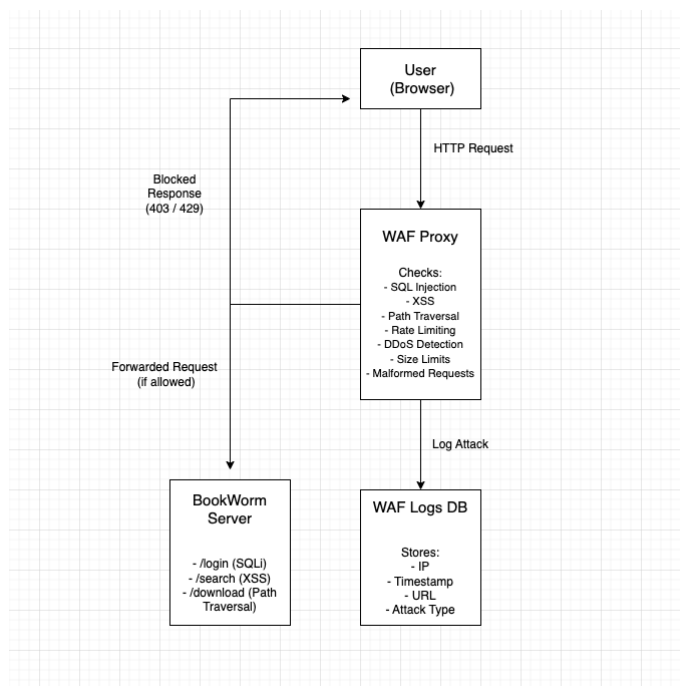
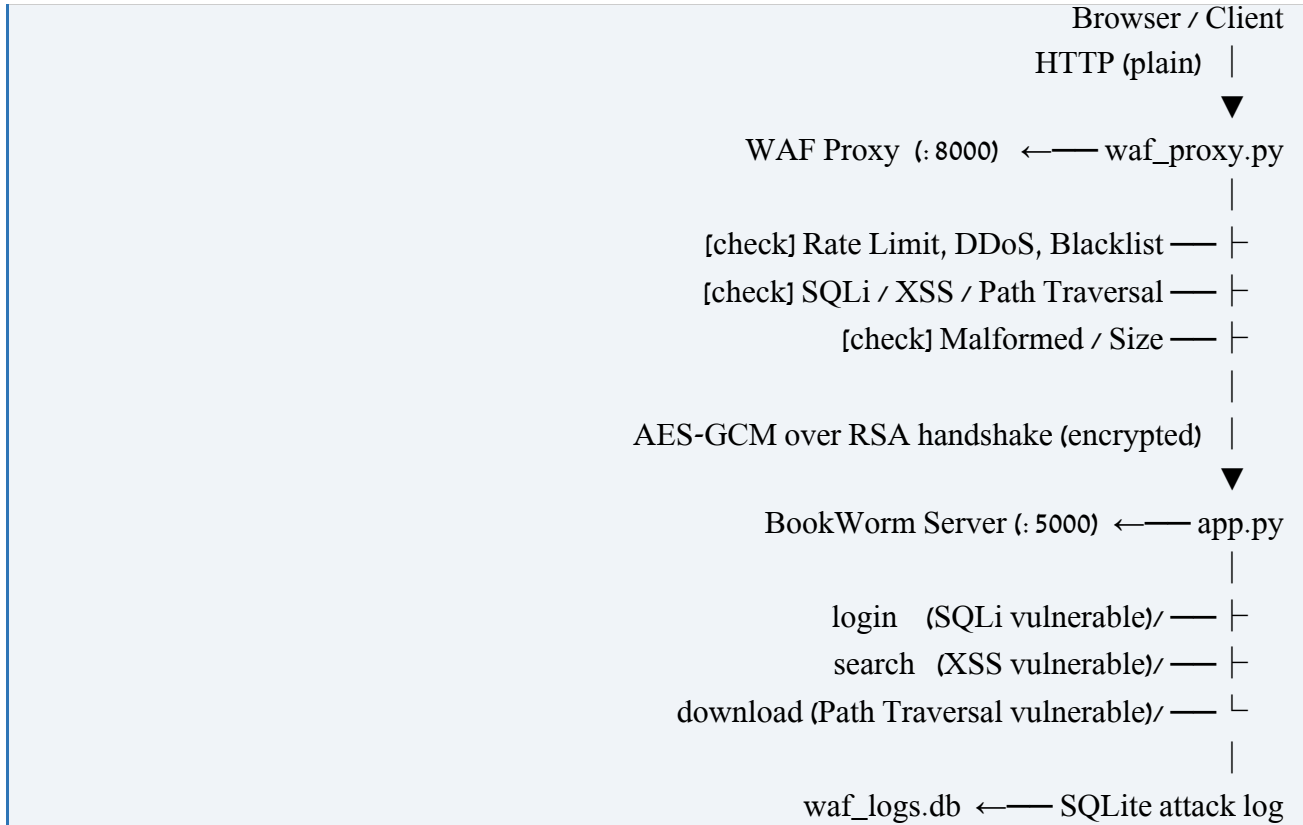
| סקריפט              | מה הוא מדמה                                                                                                       | שיטת בדיקה                          |
|---------------------|-------------------------------------------------------------------------------------------------------------------|-------------------------------------|
| attack_ratelimit.py | שולח 30 בקשות GET רצופות —<br>אמור לראות 429 אחרי 20                                                              | socket.connect + sendall<br>בלולאה  |
| attack_ddos.py      | 10 IPs שונים (11–127.0.0.2) שולחים<br>5 בקשות כל אחד, כולל 100KB body                                             | ifconfig alias + threads<br>מקבילים |
| attack_blacklist.py | שולח 4 בקשות זדוניות (SQLi, XSS,<br>Traversal, UNION SELECT) ואז<br>בקשה תקינה לבדיקה                             | socket.connect לכל בקשה             |
| attack_sizelimit.py | 9KB Header (מעל 8KB limit) ו-<br>2.1MB Body (מעל 2MB limit)                                                       | בניית HTTP raw עם padding           |
| attack_malformed.py | 7 בדיקות: garbage, method לא<br>חוקי, missing version, bad<br>,version, no Host, empty line<br>missing blank line | שליחת bytes גולמיים                 |

## מבנה / ארכיטקטורה של הפרויקט

### תיאור הארכיטקטורה

הארכיטקטורה מורכבת משני שרתים עצמאיים שמתקשרים דרך ערוץ מוצפן:

זרימת הנתונים:



## תיאור הטכנולוגיות

| קובץ                         | שימוש בפרויקט                               | טכנולוגיה / ספרייה |
|------------------------------|---------------------------------------------|--------------------|
| כולם                         | שפת הפיתוח הראשית                           | Python 3           |
| waf_proxy.py, app.py         | שרת WAF ושרת BookWorm — ללא framework       | socket (TCP)       |
| waf_proxy.py, app.py         | טיפול מקבילי בחיבורים — thread              | threading.Thread   |
| waf_proxy.py                 | זיהוי דפוסי תקיפה<br>SQLi/XSS/Traversal     | re (Regex)         |
| waf_proxy.py, http_parser.py | URL decode כפול לנרמול קלט                  | urllib.parse       |
| waf_proxy.py, app.py         | לוגים (waf_logs.db) ו-BookWorm DB (demo.db) | sqlite3            |
| waf_proxy.py                 | DDoS history window — חלון שנייה            | collections.deque  |
| crypto_utils.py              | RSA-2048 + AES-256-GCM handshake            | cryptography       |
| crypto_utils.py              | שליחה/קבלה עם header גודל                   | tcp_by_size        |
| waf_proxy_dashboard.html     | Dashboard WAF real-time                     | HTML /JS           |

## תיאור האלגוריתמים המרכזיים

אלגוריתם 1 - AnalyzeRequest (ניתוח בקשה)  
האלגוריתם הראשי שמחליט ALLOW/BLOCK לכל בקשה. פועל בתוך `handle_client` לכל לקוח ב-`thread` נפרד.

### שלבי הפעולה:

- שלב 1: בדיקת `is_ip_blacklisted(ip)` — IP blacklist. אם חסום: לוג `detected_attacks`.
- שלב 2: `receive_full_request` — קליטת הבקשה עם בדיקת גודל בזמן אמת.
- שלב 3: בדיקת `is_under_ddos_attack` + `is_verified` — DDoS. אם בהתקפה: `challenge`.
- שלב 4: `detect_rate_limiting(ip)` — בחלון 10 שניות.
- שלב 5: `double decode` + `Regex` — `is_request_suspicious(req)` על כל שדות הקלט.
- שלב 6: אם `DRY_RUN=False` ויש `detected_attacks` → חסימה. אחרת → `forward`.
- שלב 7: `RSA handshake` + `AES encrypt` — `forward_to_backend` + שליחה.

אלגוריתם 2 - DDoS Detection  
הבעיה: כיצד להבחין בין Rate Limit (IP אחד ששולח הרבה) לבין DDoS (הרבה IPs שונים)? הפתרון: `conn_history` שומר (timestamp, ip) לכל חיבור. בכל חיבור חדש מנקים ערכים ישנים מעל שנייה וסופרים unique IPs. אם מעל 5 - attack mode - ל-30 שניות.  
הפתרון הנבחר עדיף על פי Rate Limit בלבד כי הוא מגיב גם כשכל IP שולח בקשה אחת בלבד (לא נחסם ב-rate limit אבל מהווה DDoS).

אלגוריתם 3 - RSA + AES-GCM Handshake  
כל חיבור ה-WAF ל-BookWorm מוגן בנפרד. ה-WAF (client) מקבל מפתח RSA ציבורי, מייצר AES-256 אקראי, מצפין אותו עם RSA-OAEP-SHA256 ושולח לשרת. מכאן ואילך כל הודעה: IV אקראי + GCM 12B + `encrypt` → שולח `IV+tag+ciphertext`. הפתרון מספק Forward Secrecy בסיסי כי מפתח AES חדש מיוצר לכל חיבור.

## סביבת הפיתוח

| מטרה                                         | כלי                                   |
|----------------------------------------------|---------------------------------------|
| הרצת כל קבצי הפרויקט                         | +Python 3.10                          |
| ספריית הצפנה                                 | <code>pip install cryptography</code> |
| עריכת קוד                                    | PyCharm                               |
| צפייה ב- <code>waf_logs.db</code>            | SQLite Browser                        |
| צפייה ב- <code>X-WAF-Detected headers</code> | Chrome DevTools                       |

## פרוטוקול התקשורת

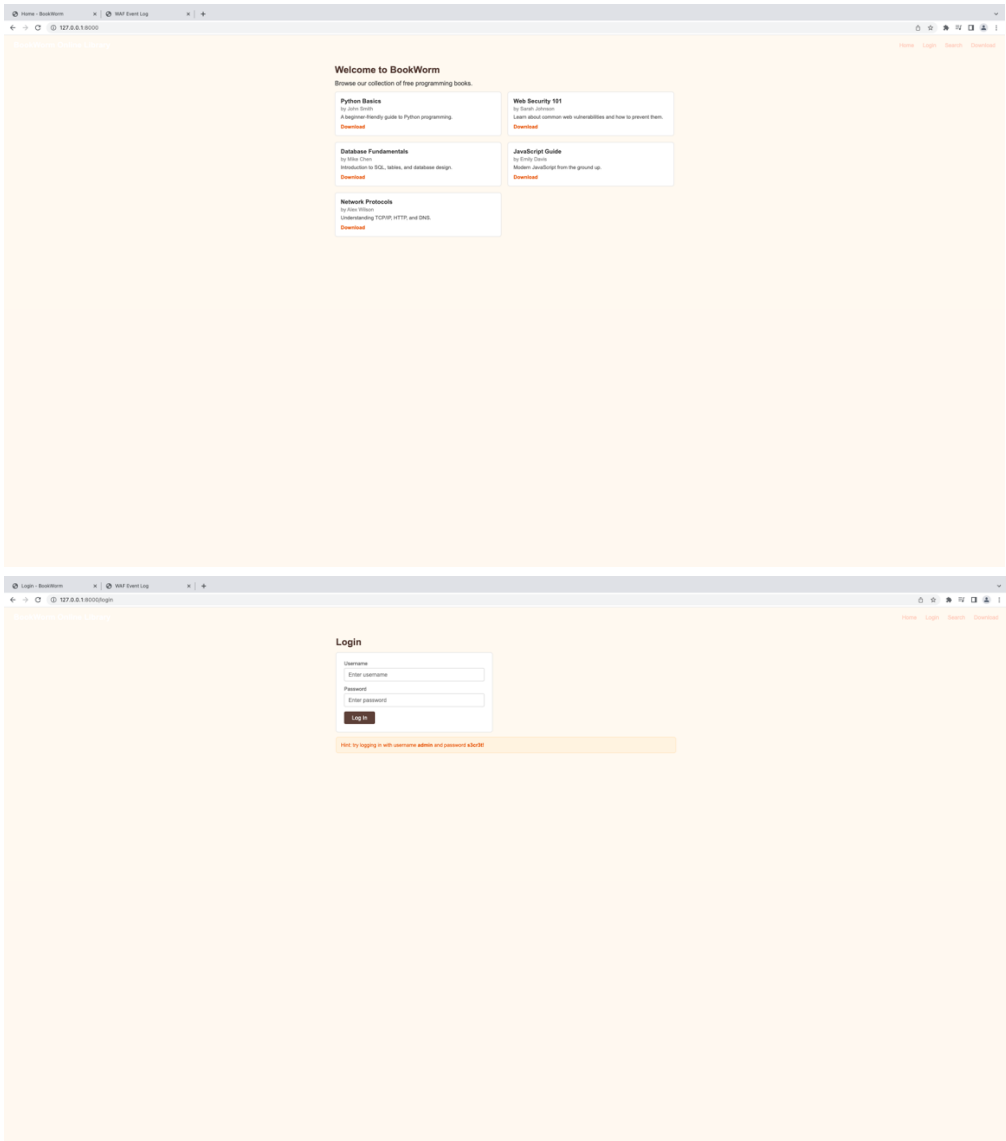
המערכת משתמשת ב-HTTP/1.1 על גבי TCP. ה-WAF מאזין על פורט 8000 ומעביר לפורט 5000.

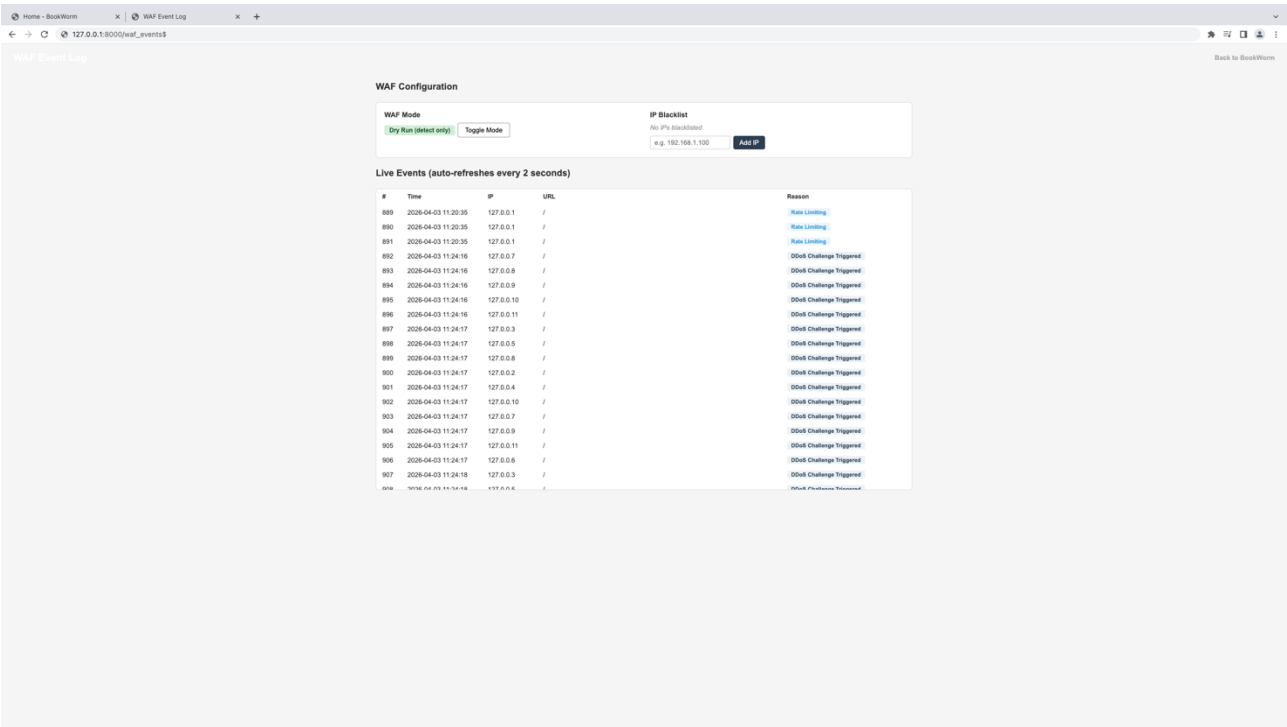
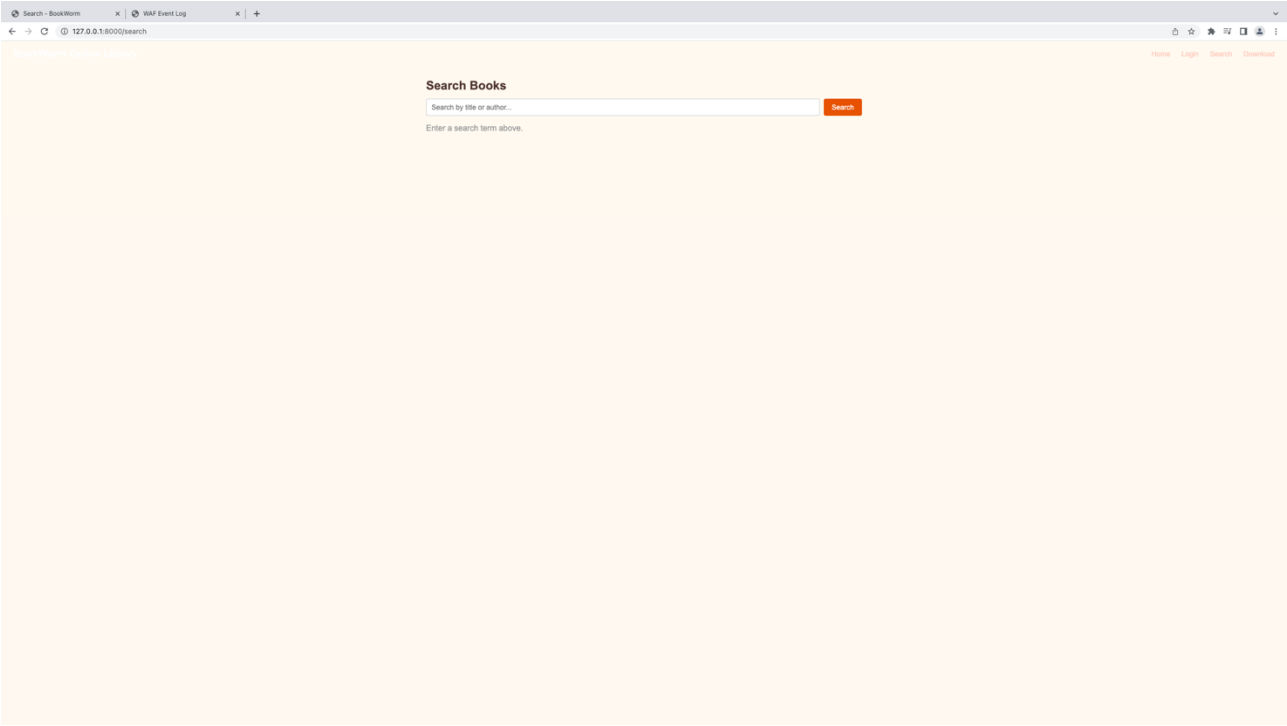
## הודעות עיקריות:

| שם ההודעה          | מ-             | אל             | תיאור                           |
|--------------------|----------------|----------------|---------------------------------|
| HTTP Request       | Browser        | WAF : 8000     | בקשת HTTP גולמית                |
| RSA Public Key     | BookWorm 5000: | WAF            | מפתח ציבורי PEM בתחילת כל חיבור |
| Encrypted AES Key  | WAF            | BookWorm 5000: | מפתח AES מוצפן ב-RSA            |
| Encrypted Request  | WAF            | BookWorm 5000: | הבקשה המקורית מוצפנת ב-AES-GCM  |
| Encrypted Response | BookWorm 5000: | WAF            | תשובת HTML/JSON מוצפנת          |
| X-WAF-Detected     | WAF            | Browser        | Header בדיקה במצב Dry Run       |
| 403/429/413/400    | WAF            | Browser        | חסימה במצב Blocking             |

תיאור מסכי המערכת

| שם המסך       | URL             | תיאור                      | מה ניתן לבצע                  |
|---------------|-----------------|----------------------------|-------------------------------|
| דף הבית       | /               | רשימת ספרים בגריד 2 עמודות | צפייה, ניווט, הורדה           |
| התחברות       | login/          | טופס Username + Password   | ניסיון כניסה (פגיע ל-SQLi)    |
| חיפוש         | =search?q/      | תיבת חיפוש + תוצאות        | חיפוש ספרים (פגיע ל-XSS)      |
| הורדה         | =download?file/ | תצוגת תוכן קובץ            | הורדת קובץ (פגיע ל-Traversal) |
| Dashboard WAF | waf_events/     | טבלת אירועים real-time     | צפייה, toggle mode, blacklist |





## תיאור מבני הנתונים

## מסד נתונים 1 - waf\_logs.db

קובץ SQLite שנוצר אוטומטית בהפעלה הראשונה דרך init\_db().

## טבלה: attack\_logs

| שם שדה    | טיפוס    | אורך | תיאור                                   | דוגמה                                   |
|-----------|----------|------|-----------------------------------------|-----------------------------------------|
| id        | INTEGER  | —    | מפתח ראשי AUTO                          | 42                                      |
| timestamp | DATETIME | —    | זמן האירוע (DEFAULT (CURRENT_TIMESTAMP) | 14: 22: 01 2025-03-15                   |
| ip        | TEXT     | 45   | כתובת IP של התוקף                       | 127.0.0.1                               |
| url       | TEXT     | 500  | ה-path שנתקף                            | login/                                  |
| reason    | TEXT     | 255  | סיבת הרישום                             | Suspicious Payload (SQLi/XSS/Traversal) |

## מסד נתונים 2 - demo.db (BookWorm)

קובץ SQLite של שרת BookWorm עם נתוני הדגמה.

## טבלה: users

| שם שדה   | טיפוס      | תיאור                       | דוגמה                  |
|----------|------------|-----------------------------|------------------------|
| id       | INTEGER PK | מזהה משתמש                  | 1                      |
| username | TEXT       | שם משתמש                    | admin                  |
| password | TEXT       | סיסמה (plain — פגיע בכוונה) | !s3cr3t                |
| email    | TEXT       | דואל                        | admin@bookworm.com     |
| role     | TEXT       | תפקיד                       | admin / user / manager |

## טבלה: books

| שם שדה      | טיפוס      | תיאור                   | דוגמה                        |
|-------------|------------|-------------------------|------------------------------|
| id          | INTEGER PK | מזהה ספר                | 1                            |
| title       | TEXT       | שם הספר                 | Python Basics                |
| author      | TEXT       | שם המחבר                | John Smith                   |
| filename    | TEXT       | שם הקובץ בתיקיית /books | python_basics.txt            |
| description | TEXT       | תיאור קצר               | ...A beginner-friendly guide |

## סקירת חולשות ואיומים

### שכבת האפליקציה:

| איום           | חולשה בBookWorm                                            | הגנת ה-WAF                               |
|----------------|------------------------------------------------------------|------------------------------------------|
| SQL Injection  | handle_login: string concatenation ישיר לשאילתה            | SQLI_PATTERNS Regex + double decode      |
| XSS            | : get_search_page<br>HTML query_text מוצג ב-<br>escape ללא | XSS_PATTERNS Regex<br>query על params    |
| Path Traversal | : handle_download<br>filename עם os.path.join<br>מהמשתמש   | PATH_TRAVERSAL_PATTERNS<br>Regex         |
| Brute Force    | אין הגבלת ניסיונות ב-<br>BookWorm                          | Rate Limiting: 20 req/10sec + Blacklist  |
| DDoS           | Flask בסיסי לא עמיד בעומס                                  | → DDoS detection: 5 IPs/sec<br>challenge |
| סיסמאות חלשות  | סיסמאות plain text ב-<br>demo.db                           | הצפנה AES-GCM בערוץ<br>WAF ↔ BookWorm    |

### שכבת התעבורה:

| איום            | תיאור                                           | הגנת ה-WAF                    |
|-----------------|-------------------------------------------------|-------------------------------|
| Malformed HTTP  | method לא חוקי,<br>missing Host, bad<br>version | error → 400 מחזיר http_parser |
| Header Overflow | Header מעל 8KB                                  | receive_full_request → 431    |
| Body Overflow   | Body מעל 2MB                                    | receive_full_request → 413    |
| MITM פנימי      | האזנה בין WAF ל-<br>BookWorm                    | AES-256-GCM + RSA handshake   |

## מימוש הפרויקט

### מודולים ומחלקות

מודולים מיובאים

| מטרה                                                             | מודול                |
|------------------------------------------------------------------|----------------------|
| listen, accept, connect, sendall, recv — TCP server/client גולמי | socket               |
| Thread לכל חיבור לקוח, Locks state משותף                         | threading            |
| Regex לזיהוי SQLi / XSS / Path Traversal                         | re                   |
| URL decode כפול (unquote_plus), פירוק query string               | urllib.parse         |
| rate_map timestamps, blacklist expiry, DDoS cooldown             | time                 |
| DDoS history — חלון שנייה גולש                                   | collections.deque    |
| waf_logs.db (WAF) ו-demo.db (BookWorm)                           | sqlite3              |
| תשובות JSON ל-Dashboard API                                      | json                 |
| קריאת Dashboard HTML, path operations ב-BookWorm                 | os                   |
| RSA-2048, AES-256-GCM — הצפנה/פענוח                              | cryptography         |
| פירוק בקשות HTTP לשדות מבניים                                    | http_parser (מקומי)  |
| RSA/AES handshake ו-send/recv מוצפן                              | crypto_utils (מקומי) |
| שליחה/קבלה עם header גודל                                        | tcp_by_size (מקומי)  |

מחלקה: HTTPRequest (http\_parser.py):  
תפקיד: מקבלת bytes גולמיים של בקשת HTTP ומפרקת אותם לשדות מבניים. מטפלת בוולידציה ובשגיאות.

### תכונות:

| שם תכונה     | טיפוס       | תפקיד                                |
|--------------|-------------|--------------------------------------|
| raw          | bytes       | הבקשה הגולמית המקורית                |
| method       | str   None  | שיטת HTTP: GET, POST, PUT, DELETE... |
| path         | str   None  | ה-path ללא query string              |
| version      | str   None  | גרסת HTTP: HTTP/1.0 או HTTP/1.1      |
| headers      | dict        | מילון headers ב-lowercase            |
| body         | str         | גוף הבקשה אחרי \n\r\n                |
| query_params | dict        | פרמטרי URL לאחר ?                    |
| form         | dict        | x-www-form-urlencoded parsed         |
| json         | dict   None | JSON body parsed                     |
| error        | str   None  | הודעת שגיאה אם הפירוק נכשל           |

## פעולות:

| שם פעולה   | פרמטרים (כניסה)   | יציאה / מה מבצעת                                            |
|------------|-------------------|-------------------------------------------------------------|
| __init__   | raw_data: bytes   | מאחסן raw ומפעיל self.parse()                               |
| parse      | —                 | מפרק request line, headers, body ; מגדיר self.error אם נכשל |
| parse_body | content_type: str | מפרק body לפי form Content-Type → json או                   |

מודול: crypto\_utils.py  
תפקיד: מספק את כל שירותי ההצפנה - יצירת מפתחות RSA, הצפנה/פענוח RSA ו-AES-GCM, וביצוע handshake.

## פעולות:

| שם פעולה                 | פרמטרים (כניסה)                 | יציאה                                   |
|--------------------------|---------------------------------|-----------------------------------------|
| generate_rsa_keypair     | —                               | (private_key, pem_public_bytes)         |
| load_public_key          | pem_bytes: bytes                | RSAPublicKey object                     |
| rsa_encrypt              | public_key, data: bytes         | ciphertext bytes (OAEP-SHA256)          |
| rsa_decrypt              | : private_key, ciphertext bytes | plaintext bytes                         |
| generate_aes_key         | —                               | 32 bytes אקראיים (AES-256)              |
| aes_encrypt              | : key: bytes, plaintext bytes   | iv(12B) + tag(16B) + ciphertext         |
| aes_decrypt              | key: bytes, data: bytes         | plaintext bytes; ValueError אם tag שגוי |
| perform_server_handshake | server_socket                   | aes_key bytes                           |
| perform_client_handshake | proxy_socket                    | aes_key bytes                           |

פונקציות מרכזיות ב-waf\_proxy.py

| שם פונקציה           | פרמטרים                               | יציאה / תיאור                                    |
|----------------------|---------------------------------------|--------------------------------------------------|
| init_db              | —                                     | יוצרת attack_logs table אם לא קיימת              |
| log_attack           | ip, url, reason                       | INSERT ל-attack_logs                             |
| receive_full_request | client_socket                         | bytes גולמיים; / HEADER_TOO_LARGE BODY_TOO_LARGE |
| send_response        | ,client_socket, body, status, ct conn | שולחת HTTP response מלא                          |
| detect_sql_injection | value: str                            | True/False — בדיקת SQL_PATTERNS                  |
| detect_xss           | value: str                            | True/False — בדיקת XSS_PATTERNS                  |

| שם פונקציה                 | פרמטרים                       | יציאה / תיאור                              |
|----------------------------|-------------------------------|--------------------------------------------|
| detect_path_traversal      | value: str                    | True/False — בדיקת PATH_TRAVERSAL_PATTERNS |
| is_request_suspicious      | : req<br>HTTPRequest          | True/False — כולל double decode            |
| detect_rate_limiting       | ip: str                       | True/False — חלון 20/10sec                 |
| record_suspicious_activity | ip: str                       | מעדכן מונה + בן אם הגיע ל-3                |
| is_ip_blacklisted          | ip: str                       | True/False — עם אוטו-reset                 |
| update_attack_state        | now: float, ip: str           | מעדכן DDoS history ו-attack_until          |
| set_verified               | ip: str                       | מוסיף ל-verified_ips עם expiry             |
| forward_to_backend         | raw_data: bytes               | bytes של תשובת BookWorm (מוצפן/מפוצח)      |
| handle_client              | ,client_socket<br>client_addr | הלולאה הראשית — keep-alive + כל הבדיקות    |
| main                       | —                             | init_db + bind(: 8000) + accept loop       |

## קטעי קוד מרכזיים

is\_request\_suspicious : זיהוי תקיפות  
הקוד הבא מהווה את ליבת זיהוי ה-payload.

```
def is_request_suspicious(req : {path, body, query_params, f... }):
    candidates = []

    if req.path:
        candidates.append(req.path)
    if req.body:
        candidates.append(req.body)

    for values in req.query_params.values():
        candidates.extend(values)

    for values in req.form.values():
        candidates.extend(values)

    for value in candidates:
        if not isinstance(value, str):
            continue

        decoded_value = urllib.parse.unquote_plus(value)
        double_decoded = urllib.parse.unquote_plus(decoded_value)
        normalized_value = double_decoded.lower()

        if detect_sql_injection(normalized_value) or detect_sql_injection(value):
            return True
        if detect_xss(normalized_value) or detect_xss(value):
            return True
        if detect_path_traversal(normalized_value) or detect_path_traversal(value):
            return True

    return False
```

handle\_client : הלולאה הראשית עם keep-alive  
 הלולאה תומכת ב-HTTP keep-alive - ממשיכה לקבל בקשות על אותו socket עד שהלקוח שולח  
 Connection: close או עד timeout של 30 שניות :

```
def handle_client(client_socket : {settimeout, recv, timeout, ...}, client_addr : {__getitem__}):
    try:
        ip = client_addr[0]
        client_socket.settimeout(30) # timeout לחיבורי keep-alive

        while True:
            detected_attacks = []
            if is_ip_blacklisted(ip):
                log_attack(ip, uri: "-", reason: "IP Blacklisted")
                detected_attacks.append("IP Blacklisted")
            request_data = receive_full_request(client_socket)
            if not request_data:
                return

            # waf control api
            if request_data.startswith(b"GET /waf_control?secret=supersecretwafkey"):
                handle_waf_control(request_data, client_socket)
                continue

            # dashboard page
            if request_data.startswith(b"GET /waf_events"):
                handle_dashboard(request_data, client_socket)

            if request_data == b"HEADER_TOO_LARGE":
                log_attack(ip, uri: "-", reason: "Header Too Large")
                send_error_response(client_socket, status: "431 Request Header Fields Too Large", body: "Header Too Large")
                break

            if request_data == b"BODY_TOO_LARGE":
                log_attack(ip, uri: "-", reason: "Body Too Large")
                send_error_response(client_socket, status: "413 Payload Too Large", body: "Body Too Large")
                break

            req = HTTPRequest(request_data)

            if req.error:
                path = getattr(req, "path", "-")
                if not path:
                    path = "-"
                log_attack(ip, path, reason: f"Bad Request: {req.error}")
                detected_attacks.append(f"Bad Request: {req.error}")

            # DDoS / rate limit / security checks
            action = run_waf_checks(req, ip, client_socket, detected_attacks)
            if action == "break":
                break
            elif action == "continue":
                continue

            # block or pass through
            action = apply_waf_policy(detected_attacks, client_socket)
            if action == "break":
                break
            elif action == "continue":
                continue

            # forward to backend
            backend_response = forward_to_backend(request_data)
            # header יש מתקפה ולא מוסמים הופסת
            backend_response = add_waf_debug_header(backend_response, detected_attacks)
            client_socket.sendall(backend_response)

            # check if client wants to close
            conn = req.headers.get("connection", "")
            if conn.lower() == "close":
                break

        finally:
            client_socket.close()
```

forward\_to\_backend: RSA+AES handshake  
כל חיבור ל-BookWorm מוצפן

```
def forward_to_backend(raw_data : {socket, AF_INET, SOCK_STREAM...}):
    backend = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    try:
        backend.connect(("127.0.0.1", 5000))

        # TLS handshake עם ה backend
        aes_key = crypto_utils.perform_client_handshake(backend)

        # הצפנה ושליחה
        encrypted_req = crypto_utils.aes_encrypt(aes_key, raw_data)
        crypto_utils.send_msg(backend, encrypted_req)

        # קבלה ופענוח
        encrypted_resp = crypto_utils.recv_msg(backend)
        if encrypted_resp:
            return crypto_utils.aes_decrypt(aes_key, encrypted_resp)
        return b""

    except Exception as e:
        #print("DEBUG forward:", raw_data[:80])
        print(f"Backend Connection/TLS Error: {e}")
        return build_response_str( body: "Backend SSL/TLS Handshake Error or Backend Down", status="502 Bad Gateway")

    finally:
        try:
            backend.close()
        except Exception:
            pass
```

```
def update_attack_state(now : {__sub__}, ip):
    global attack_until

    with conn_lock:
        conn_history.append((now, ip))

        # מוחקים חיבורים ישנים יותר משנייה
        while conn_history and now - conn_history[0][0] > 1:
            conn_history.popleft()

        # סופרים כמה IP-ים שונים התחברו בשנייה האחרונה
        unique_ips = set()
        for t, addr in conn_history:
            unique_ips.add(addr)

        # DDoS ששולח הרבה IP ים שונים, לא רק IP-ים אחדים
        if len(unique_ips) > max_ips_per_sec:
            if now >= attack_until:
                print(f"[DDoS] Under Attack! {len(unique_ips)} unique IPs/sec")
                attack_until = now + ATTACK_COOLDOWN_SEC
```

## מסמך בדיקות

| #  | מטרה                   | קלט                             | פלט צפוי       | תוצאה בפועל              | בעיות ופתרון                       |
|----|------------------------|---------------------------------|----------------|--------------------------|------------------------------------|
| 1  | בקשה תקינה             | / GET                           | 200 ALLOW      | OK 200 ✓                 | —                                  |
| 2  | SQLi בסיסי             | ' -- OR 1=1                     | 403 BLOCK      | 403 ✓                    | —                                  |
| 3  | SQLi UNION             | ' - UNION SELECT * FROM users - | 403 BLOCK      | 403 ✓                    | —                                  |
| 4  | XSS script tag         | <script>alert('xss')</script>   | 403 BLOCK      | 403 ✓                    | —                                  |
| 5  | XSS event handler      | onerror=alert(1)                | 403 BLOCK      | 403 ✓                    | —                                  |
| 6  | Path Traversal plain   | etc/passwd/../../               | 403 BLOCK      | 403 ✓                    | —                                  |
| 7  | Path Traversal encoded | %2e%2e%2f%2e%2e%2fetc/passwd    | 403 BLOCK      | 403 ✓<br>(double decode) | ראשון נכשל —<br>נוסף double decode |
| 8  | Rate Limit             | 30 בקשות רצופות                 | 429 אחרי 20    | at 429 ✓<br>#21          | —                                  |
| 9  | DDoS                   | 10 IPs במקביל                   | Challenge HTML | ✓<br>Challenge           | דרוש sudo ifconfig                 |
| 10 | IP Blacklist           | 3 התקפות → נורמלי               | 403 לנורמלי    | 403 ✓                    | —                                  |
| 11 | Header 9KB             | X-Padding: A*9000               | 431            | 431 ✓                    | —                                  |
| 12 | Body 2.1MB             | POST body 2.1MB                 | 413            | 413 ✓                    | —                                  |
| 13 | Method לא חוקי         | BLAH / HTTP/1.1                 | 400            | 400 ✓                    | —                                  |
| 14 | Missing Host           | GET / HTTP/1.1 (no Host)        | 400            | 400 ✓                    | —                                  |
| 15 | עברית / Unicode        | comment=שלום עולם               | 200 ALLOW      | ✓<br>ALLOW               | —                                  |
| 16 | False Positive XSS     | הסבר המכיל <script> כטקסט       | BLOCK שגוי     | נפתר                     | הידוק pattern                      |

## מדריך למשתמש

### עץ קבצי המערכת

```

/WAF_Project
  waf_proxy.py — | # שרת ה-WAF הראשי (פורט 8000)
  demo_app.py — | # שרת BookWorm הפגיע (פורט 5000)
  http_parser.py — | # מחלקת HTTPRequest לפירוק בקשות
  crypto_utils.py — | # RSA + AES-GCM handshake utilities
  tcp_by_size.py — | # שליחה/קבלה עם header גודל
  waf_proxy_dashboard.html — | # Dashboard
  waf_logs.db — | # SQLite log
  demo.db — | # SQLite BookWorm DB (נוצר אוטומטית)
  attack_ratelimit.py — | # בדיקת Rate Limiting
  attack_ddos.py — | # בדיקת DDoS Detection
  attack_blacklist.py — | # בדיקת IP Blacklisting
  attack_sizelimit.py — | # בדיקת Size Limits
  attack_malformed.py — | # בדיקת Malformed Requests
  /www — |
  /books — | # קבצי ספרים (txt.)

```

### התקנת המערכת

#### דרישות סביבה:

- Python 3.8 ומעלה
- מערכת הפעלה: Windows / Linux / macOS
- פורט 8000 ו-5000 פנויים

#### התקנת תלויות:

```
pip install cryptography
```

#### הפעלת המערכת — שלבים:

```

# טרמינל 1 — הפעל שרת BookWorm
python app.py
# פלט: BookWorm server running on port 5000

# טרמינל 2 — הפעל WAF
python waf_proxy.py
# פלט: WAF running on 8000

# גלוש לאתר דרך ה-WAF:
/http://127.0.0.1:8000

# Dashboard ניהולי:
http://127.0.0.1:8000/waf_events

```

## אופן שימוש לפי משתמש

מנהל מערכת

| פעולה                  | איך מבצעים                                           |
|------------------------|------------------------------------------------------|
| כניסה ל-Dashboard      | <code>http://127.0.0.1:8000/waf_events</code>        |
| שינוי Dry Run Blocking | לחצן Toggle Mode ב-Dashboard (או משתנה DRY_RUN בקוד) |
| צפייה באירועי אבטחה    | טבלת Live Events — מתעדכנת כל 2 שניות                |
| הוספת IP לחסימה ידנית  | הזנת IP + לחצן Add IP ב-Dashboard                    |
| הסרת IP מחסימה         | לחצן Remove ליד ה-IP ברשימת Blacklist                |
| ייצוא לוגים            | ישירות מ-waf_logs.db דרך SQLite Browser              |

בדיקות מהדפדפן

| סוג תקיפה      | URL לפתיחה                                  | קלט לניסוי                                                   |
|----------------|---------------------------------------------|--------------------------------------------------------------|
| SQL Injection  | <code>http://127.0.0.1:8000/login</code>    | :Username: ' OR 1=1 -- Password כלשהו                        |
| UNION SQLi     | <code>http://127.0.0.1:8000/login</code>    | * Username: ' UNION SELECT -- FROM users                     |
| XSS            | <code>http://127.0.0.1:8000/search</code>   | <code>&lt;q=&lt;script&gt;alert('XSS')&lt;/script&gt;</code> |
| Path Traversal | <code>http://127.0.0.1:8000/download</code> | <code>file=../../secrets.txt?</code>                         |

הרצת סקריפטי בדיקה

| סקריפט        | פקודה                                   | מה לצפות                                        |
|---------------|-----------------------------------------|-------------------------------------------------|
| Rate Limiting | <code>python attack_ratelimit.py</code> | שורות 1-20 : OK, שורות 21-30 : RATE 429 LIMITED |
| DDoS          | <code>sudo python attack_ddos.py</code> | בקשות מקבלות Challenge HTML                     |
| IP Blacklist  | <code>python attack_blacklist.py</code> | attack 1-3: BLOCK, normal request: 403          |
| Size Limits   | <code>python attack_sizelimit.py</code> | Header 9KB: 431, Body 2.1MB: 413                |
| Malformed     | <code>python attack_malformed.py</code> | כל 7 הבדיקות : BLOCKED (400)                    |

## רפלקציה

### תהליך העבודה

פרויקט ה-WAF היה מהאתגרים הטכניים והמעניינים שביצעתי. ההחלטה לבנות שרת בתקשורת של פרוטוקול HTTP - להבין לעומק כיצד פרוטוקול HTTP עובד: כיצד מגיעים bytes, כיצד מפרקים header מ-body, וכיצד מנהלים keep-alive על אותו socket.

בתחילה, תכנתי WAF שבדק רק את ה-URL. לאחר ניסויים ראשונים גיליתי שניתן בקלות לעקוף אותו על ידי שליחת payload ב-body POST או ב-form. זה לימד אותי שמערכת הגנה חייבת לבדוק את כל שדות הקלט.

### תהליך למידה

הדברים החדשים שלמדתי במהלך הפרויקט:

- **Regex** מתקדם: שימוש ב-\b לגבולות מילה, s\+ לרווחים גמישים. ההבדל בין ביטוי רחב מדי (שחוסם שגוי) לצר מדי (שפספס תקיפה).
- **Double URL Encoding**: תוקף יכול לכתוב %2527 במקום ' (25% = % ואז 27 = '). הפתרון: decode כפול לפני בדיקה.
- **RSA + AES Hybrid Encryption**: למה לא להשתמש רק ב-RSA? כי RSA איטי ומוגבל בגודל. הפתרון הנכון: RSA להחלפת מפתח, AES להצפנת הנתונים.
- **AES-GCM**: מבטיח גם סודיות (encryption) וגם שלמות (authentication tag). אם ה-tag שגוי — פענוח נכשל עם ValueError.
- **threading + Locks**: כשיש כמה threads שמשתמשים באותו dict (rate\_map, blacklist\_map), חייבים Lock כדי למנוע race conditions.
- **DDoS vs Rate Limit**: ההבדל הקריטי — Rate Limit מגן מפני IP אחד, DDoS מגן מפני רבים. השתמשתי ב-deque לחלון זמן יעיל.

### תובנות מהתהליך

- הדבר שהפתיע אותי ביותר: כמה שינוי קטן בקלט (כמו %2e%2e במקום ..) יכול לעקוף חוק Regex שנראה חזק. זה הבהיר לי מדוע מערכות WAF מסחריות משתמשות גם ב-Machine Learning - לא ניתן לכסות את כל הגרסאות האפשריות רק בחוקים סטטיים.
- הדבר השני שלמדתי: בדיקות הן הכרחיות. ה-False Positive (חסימת טקסט הסבר שהכיל <script>) לימד אותי שכל שינוי ב-Regex דורש regression testing.
- קיבלתי עזרה ממורה המנחה בהבנת מנגנון ה-DDoS Detection ובמימוש ה-keep-alive לחיבורים. בנוסף, תיעוד ספריית cryptography ו-Stack Overflow היו משאבים חיוניים.

### שיפורים עם משאבים נוספים

- תמיכה ב-HTTPS (SSL/TLS) בין הדפדפן ל-WAF.
- ממשק ניהול עשיר עם גרפים, ואפשרות לעדכון חוקים בזמן ריצה.

## ביבליוגרפיה

### [Regex בפיתון](#)

DEV Community - מאמר של מפתח שבנה WAF בפיתון מאפס. השתמשתי במאמר זה להבנת הגישה המעשית למימוש WAF - בפרט בנושאי סריקת body של בקשות POST, זיהוי SQLi ו-XSS באמצעות Regex, Rate Limiting. המאמר עוזר לי לאמת שהארכיטקטורה שבחרתי הולכת בכיוון הנכון. [DEV Community](#)

Cloudflare Learning - מקור תיאורטי שקראתי בתחילת הפרויקט להבנת הבסיס. המאמר הסביר מהו WAF, כיצד הוא פועל כ-Reverse Proxy, וההבדל בין מודל Blocklist למודל Allowlist. [Cloudflare Learning](#)

# Table of Contents

---

|                                     |    |
|-------------------------------------|----|
| <a href="#">attack_blacklist.py</a> | 2  |
| <a href="#">attack_ddos.py</a>      | 4  |
| <a href="#">attack_malformed.py</a> | 6  |
| <a href="#">attack_ratelimit.py</a> | 8  |
| <a href="#">attack_sizelimit.py</a> | 9  |
| <a href="#">crypto_utils.py</a>     | 11 |
| <a href="#">demo_app.py</a>         | 13 |
| <a href="#">http_parser.py</a>      | 22 |
| <a href="#">tcp_by_size.py</a>      | 24 |
| <a href="#">waf_proxy.py</a>        | 25 |

```

# ■■■■■ IP blacklisting
# ■■■■ ■■■■■ ■■■■■■ ■■■ ■■■■■ ■-WAF ■■■■■ ■■ ■-IP ■■■■
# 3 ■■■■■ = ■■■ ■-5 ■■■■

import socket
import time

waf_addr = ("127.0.0.1", 8000)

# ■■■■■ ■■■■■■ ■■■■■
bad_requests = [
    "GET /login?username=' OR 1=1 -- HTTP/1.1\r\nHost: 127.0.0.1:8000\r\nConnection: close\r\n\r\n",
    "GET /search?q=<script>alert('xss')</script> HTTP/1.1\r\nHost: 127.0.0.1:8000\r\nConnection: close\r\n\r\n",
    "GET /download?file=../../etc/passwd HTTP/1.1\r\nHost: 127.0.0.1:8000\r\nConnection: close\r\n\r\n",
    "GET /login?username=' UNION SELECT * FROM users -- HTTP/1.1\r\nHost: 127.0.0.1:8000\r\nConnection: close\r\n\r\n",
]

normal_req = "GET / HTTP/1.1\r\nHost: 127.0.0.1:8000\r\nConnection: close\r\n\r\n"

def send_request(raw_http):
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(5)
        s.connect(waf_addr)
        s.sendall(raw_http.encode())

        resp = b""
        while True:
            d = s.recv(4096)
            if not d:
                break
            resp += d
        s.close()

        line = resp.decode(errors="replace").split("\r\n")[0]
        parts = line.split(" ", 2)
        code = int(parts[1])
        msg = parts[2] if len(parts) > 2 else ""
        return code, msg

    except Exception as e:
        return -1, str(e)

def main():
    print("IP Blacklist Test")
    print(f"target: {waf_addr[0]}:{waf_addr[1]}")
    print("3 offenses = banned for 5 min")
    print()

    # ■■■■ 1 - ■■■■ ■■■■■ ■■■■■■
    print("sending malicious requests...")
    for i, payload in enumerate(bad_requests, 1):
        code, msg = send_request(payload)
        print(f" attack {i}: HTTP {code} {msg}")
        time.sleep(0.3)

```

```
print()

# 2 - -IP
print("checking if we got banned...")
time.sleep(0.5)

code, msg = send_request(normal_req)
print(f" normal request: HTTP {code} {msg}")
print()

if code == 403:
    print("IP got blacklisted! even normal requests blocked")
elif code == 200:
    print("request went through - WAF probably in dry-run mode")
else:
    print(f"unexpected status {code}")

if __name__ == "__main__":
    main()
```

```

# ■■■■■■■■■■ DDoS - ■■■■ ■■■■■■ ■■■■■■ IP
# ■■■■ ■■■■■■ ■■ sudo ■■■■■■ ■-IP-■■ ■■■■■■■■

import socket
import time
import threading
import os
import sys

TARGET = ("127.0.0.1", 8000)
NUM_ATTACKERS = 10
REQS_EACH = 5
BODY_SIZE = 100000 # 100KB ■■■■ ■■■■■■

print_lock = threading.Lock()
stats = {"ok": 0, "challenged": 0, "errors": 0}

fake_ips = [f"127.0.0.{i}" for i in range(2, 2 + NUM_ATTACKERS)]

def setup_fake_ips():
    print(f"■■■■■ {NUM_ATTACKERS} ■■■■■■ IP (■■■■■ sudo)...")
    for ip in fake_ips:
        ret = os.system(f"sudo ifconfig lo0 alias {ip} up 2>/dev/null")
        if ret != 0:
            print(f"■■■■■ ■■■■■■ {ip} - ■■■■ ■■ sudo")
            return False
    print(f"IPs ready: {fake_ips[0]} to {fake_ips[-1]}")
    return True

def cleanup_fake_ips():
    for ip in fake_ips:
        os.system(f"sudo ifconfig lo0 -alias {ip} 2>/dev/null")

def flood_from_ip(src_ip):
    for n in range(REQS_EACH):
        try:
            s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            s.settimeout(10)
            s.bind((src_ip, 0))
            s.connect(TARGET)

            body = "A" * BODY_SIZE
            http_req = (
                "POST / HTTP/1.1\r\n"
                f"Host: {TARGET[0]}:{TARGET[1]}\r\n"
                "Content-Type: text/plain\r\n"
                f"Content-Length: {len(body)}\r\n"
                "Connection: close\r\n"
                "\r\n" + body
            )
            s.sendall(http_req.encode())

            data = b""
            while True:
                chunk = s.recv(4096)
                if not chunk:
                    break
                data += chunk
            s.close()

```

```
        text = data.decode(errors="replace").lower()
        with print_lock:
            if "challenge" in text or "security check" in text:
                stats["challenged"] += 1
                print(f" {src_ip} #{n+1}: CHALLENGE")
            else:
                stats["ok"] += 1
                print(f" {src_ip} #{n+1}: OK")

    except Exception as e:
        with print_lock:
            stats["errors"] += 1
            print(f" {src_ip} #{n+1}: ERROR - {e}")

def main():
    if not setup_fake_ips():
        sys.exit(1)

    total = NUM_ATTACKERS * REQS_EACH
    print()
    print(f"DDoS Flood - {NUM_ATTACKERS} attackers, {REQS_EACH} reqs each")
    print(f"payload size: {BODY_SIZE // 1000}KB, total: {total} connections")
    print()

    threads = []
    t0 = time.time()

    for ip in fake_ips:
        t = threading.Thread(target=flood_from_ip, args=(ip,))
        threads.append(t)
        t.start()
        time.sleep(0.02)

    for t in threads:
        t.join()

    elapsed = time.time() - t0
    print()
    print(f"done in {elapsed:.2f}s")
    print(f" OK: {stats['ok']}")
    print(f" Challenged: {stats['challenged']}")
    print(f" Errors: {stats['errors']}")
    if stats["challenged"] > 0:
        print("DDoS detection worked!")
    else:
        print("no challenges - maybe WAF is in dry-run?")

    cleanup_fake_ips()

if __name__ == "__main__":
    main()
```

```

# ##### HTTP #####
# -WAF ##### 400

import socket

WAF = ("127.0.0.1", 8000)

# (##, data ##)
tests = [
    ("garbage data", b"asjkdfhaskjdfh\r\n\r\n"),
    ("invalid method", b"BLAH / HTTP/1.1\r\nHost: 127.0.0.1:8000\r\n\r\n"),
    ("missing HTTP version", b"GET /\r\nHost: 127.0.0.1:8000\r\n\r\n"),
    ("bad HTTP version", b"GET / HTTP/9.9\r\nHost: 127.0.0.1:8000\r\n\r\n"),
    ("no Host header", b"GET / HTTP/1.1\r\n\r\n"),
    ("empty request line", b"\r\nHost: 127.0.0.1:8000\r\n\r\n"),
    ("missing blank line at end", b"GET / HTTP/1.1\r\nHost: 127.0.0.1:8000"),
]

def send_broken_request(data):
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(5)
        s.connect(WAF)
        s.sendall(data)

        buf = b""
        while True:
            chunk = s.recv(4096)
            if not chunk:
                break
            buf += chunk
        s.close()

        if not buf:
            return 0, "no response"

        line1 = buf.split(b"\r\n")[0].decode(errors="replace")
        p = line1.split(" ", 2)
        return int(p[1]) if len(p) >= 2 else 0, p[2] if len(p) > 2 else ""

    except Exception as e:
        return -1, str(e)

def main():
    print("Malformed Request Test")
    print()

    ok = 0
    fail = 0

    for i, (name, data) in enumerate(tests, 1):
        code, msg = send_broken_request(data)

        if code == 400:
            ok += 1
            res = "BLOCKED (400)"
        elif code == 200:
            fail += 1
            res = "ALLOWED - probably dry-run"
        elif code == 0:

```

```
        ok += 1
        res = "connection dropped"
    else:
        res = f"HTTP {code} {msg}"

    print(f"    {i}. {name}")
    print(f"        -> {res}")
    print()

    print(f"blocked: {ok}    allowed: {fail}")

if __name__ == "__main__":
    main()
```

```
# ##### rate limiting
# ##### 30 #####, #-WAF ##### 20

import socket

HOST = "127.0.0.1"
PORT = 8000
NUM_REQUESTS = 30

def main():
    print("Rate Limit Test")
    print("sending", NUM_REQUESTS, "requests...")
    print()

    allowed = 0
    blocked = 0

    for i in range(1, NUM_REQUESTS + 1):
        try:
            s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            s.settimeout(5)
            s.connect((HOST, PORT))
            s.sendall(b"GET / HTTP/1.1\r\nHost: 127.0.0.1:8000\r\nConnection: close\r\n\r\n")

            resp = b""
            while True:
                part = s.recv(4096)
                if not part:
                    break
                resp += part
            s.close()

            first_line = resp.split(b"\r\n")[0].decode(errors="replace")
            code = int(first_line.split(" ")[1])

            if code == 429:
                blocked += 1
                print(f" #{i} -> 429 RATE LIMITED")
            else:
                allowed += 1
                print(f" #{i} -> {code} OK")
        except:
            print(f" #{i} -> ERROR")

    print()
    print(f"allowed: {allowed} blocked: {blocked}")

if __name__ == "__main__":
    main()
```

```

# ■■■■■ ■■■■■ ■■■ ■■■■
# WAF limits: headers 8KB, body 2MB

import socket

server = ("127.0.0.1", 8000)

def try_send(raw_bytes):
    """■■■■ status code"""
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(10)
        s.connect(server)
        s.sendall(raw_bytes)

        buf = b""
        while True:
            chunk = s.recv(4096)
            if not chunk:
                break
            buf += chunk
        s.close()

        decoded = buf.decode(errors="replace")
        parts = decoded.split("\r\n")[0].split(" ", 2)
        return int(parts[1]), parts[2] if len(parts) > 2 else ""

    except Exception as e:
        return -1, str(e)

def test_big_header():
    print("test 1 - header 9KB (limit 8KB)")

    padding = "A" * 9000
    req = (
        "GET / HTTP/1.1\r\n"
        "Host: 127.0.0.1:8000\r\n"
        "X-Padding: " + padding + "\r\n"
        "\r\n"
    )

    code, msg = try_send(req.encode())

    if code in (413, 400, 431):
        print(f" -> HTTP {code} {msg}")
        print(" blocked correctly!")
    elif code == -1:
        print(f" -> connection dropped: {msg}")
    elif code == 200:
        print(f" -> HTTP {code} - went through (dry-run?)")
    else:
        print(f" -> HTTP {code} {msg}")

def test_big_body():
    print()
    print("test 2 - body 2.1MB (limit 2MB)")

    big_body = "X" * (2 * 1024 * 1024 + 100000)
    req = (

```

```
        "POST / HTTP/1.1\r\n"
        "Host: 127.0.0.1:8000\r\n"
        "Content-Type: text/plain\r\n"
        f"Content-Length: {len(big_body)}\r\n"
        "\r\n" + big_body
    )

    code, msg = try_send(req.encode())

    if code in (413, 400):
        print(f"  -> HTTP {code} {msg}")
        print("  blocked correctly!")
    elif code == -1:
        print(f"  -> connection dropped: {msg}")
    elif code == 200:
        print(f"  -> HTTP {code} - went through (dry-run?)")
    else:
        print(f"  -> HTTP {code} {msg}")

def main():
    print(f"Size Limit Test - target {server[0]}:{server[1]}")
    print()
    test_big_header()
    test_big_body()
    print()
    print("check waf events: http://127.0.0.1:8000/waf_events")

if __name__ == "__main__":
    main()
```

```
import os
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import tcp_by_size

def generate_rsa_keypair():
    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048,
    )
    public_key = private_key.public_key()

    pem_public = public_key.public_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PublicFormat.SubjectPublicKeyInfo
    )
    return private_key, pem_public

def load_public_key(pem_bytes):
    return serialization.load_pem_public_key(pem_bytes)

def rsa_encrypt(public_key, data):
    return public_key.encrypt(
        data,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )

def rsa_decrypt(private_key, ciphertext):
    return private_key.decrypt(
        ciphertext,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )

def generate_aes_key():
    return os.urandom(32)

# ■■■■■ AES-GCM
def aes_encrypt(key, plaintext):
    iv = os.urandom(12)
    cipher = Cipher(algorithms.AES(key), modes.GCM(iv))
    encryptor = cipher.encryptor()
    ciphertext = encryptor.update(plaintext) + encryptor.finalize()
    return iv + encryptor.tag + ciphertext

# ■■■■■
def aes_decrypt(key, data):
    if len(data) < 28:
        raise ValueError("Encrypted data is too short")
    iv = data[:12]
    tag = data[12:28]
    ciphertext = data[28:]
```

```
cipher = Cipher(algorithms.AES(key), modes.GCM(iv, tag))
decryptor = cipher.decryptor()
return decryptor.update(ciphertext) + decryptor.finalize()

def send_msg(sock, msg):
    tcp_by_size.send_with_size(sock, msg)

def recv_msg(sock):
    return tcp_by_size.recv_by_size(sock)

# handshake - ■■■ ■■■■
def perform_server_handshake(server_socket):
    private_key, pem_public = generate_rsa_keypair()
    send_msg(server_socket, pem_public)

    encrypted_aes_key = recv_msg(server_socket)
    if not encrypted_aes_key:
        raise ConnectionError("No AES key received from Proxy")

    aes_key = rsa_decrypt(private_key, encrypted_aes_key)
    return aes_key

# handshake - ■■■ ■■■■
def perform_client_handshake(proxy_socket):
    pem_public = recv_msg(proxy_socket)
    if not pem_public:
        raise ConnectionError("No Public Key received from Backend")
    public_key = load_public_key(pem_public)

    aes_key = generate_aes_key()
    encrypted_aes_key = rsa_encrypt(public_key, aes_key)

    send_msg(proxy_socket, encrypted_aes_key)
    return aes_key
```

```

# BookWorm - 
# 5000, -WAF 

import socket
import threading
import sqlite3
import os
import html as html_mod
import urllib.parse
from http_parser import HTTPRequest
import crypto_utils

DEMO_DB = "demo.db"
BOOKS_DIR = os.path.join("www", "books")
# todo: maybe add categories later

def init_demo_db():
    conn = sqlite3.connect(DEMO_DB)
    cursor = conn.cursor()

    # users
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY,
            username TEXT,
            password TEXT,
            email TEXT,
            role TEXT
        )
    """)

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS books (
            id INTEGER PRIMARY KEY,
            title TEXT,
            author TEXT,
            filename TEXT,
            description TEXT
        )
    """)

    cursor.execute("SELECT COUNT(*) FROM users")
    if cursor.fetchone()[0] == 0:
        cursor.executemany(
            "INSERT INTO users (username, password, email, role) "
            "VALUES (?, ?, ?, ?)",
            [
                ("admin", "s3cr3t!", "admin@bookworm.com", "admin"),
                ("john_doe", "password1", "john@bookworm.com", "user"),
                ("sarah_k", "letmein", "sarah@bookworm.com", "user"),
                ("mike_r", "qwerty99", "mike@bookworm.com", "manager"),
                ("test_user", "test123", "test@bookworm.com", "user"),
            ]
        )

    cursor.execute("SELECT COUNT(*) FROM books")
    if cursor.fetchone()[0] == 0:
        cursor.executemany(
            "INSERT INTO books (title, author, filename, description) "
            "VALUES (?, ?, ?, ?)",
            [

```

```

        ("Python Basics",
         "John Smith",
         "python_basics.txt",
         "A beginner-friendly guide to Python programming."),
        ("Web Security 101",
         "Sarah Johnson",
         "web_security_101.txt",
         "Learn about common web vulnerabilities and how to prevent them."),
        ("Database Fundamentals",
         "Mike Chen",
         "database_fundamentals.txt",
         "Introduction to SQL, tables, and database design."),
        ("JavaScript Guide",
         "Emily Davis",
         "javascript_guide.txt",
         "Modern JavaScript from the ground up."),
        ("Network Protocols",
         "Alex Wilson",
         "network_protocols.txt",
         "Understanding TCP/IP, HTTP, and DNS."),
    ]

    conn.commit()
    conn.close()

def build_response(body_text, content_type="text/html; charset=utf-8"):
    body = body_text.encode("utf-8") if isinstance(body_text, str) else body_text
    header = (
        "HTTP/1.1 200 OK\r\n"
        "Content-Type: " + content_type + "\r\n"
        "Content-Length: " + str(len(body)) + "\r\n"
        "\r\n"
    )
    return header.encode("utf-8") + body

SITE_CSS = """
* { margin: 0; padding: 0; box-sizing: border-box; }
body { font-family: Arial, sans-serif; background: #fff8f0; color: #333; }

/* top bar */
.topbar {
    background: #5D4037; color: white;
    padding: 14px 24px; display: flex;
    align-items: center; justify-content: space-between;
}
.topbar .logo { font-size: 20px; font-weight: bold; }
.topbar nav a {
    color: #ffccbc; text-decoration: none;
    margin-left: 18px; font-size: 14px;
}
.topbar nav a:hover { color: white; }

/* main */
.container { max-width: 900px; margin: 24px auto; padding: 0 16px; }

h2 { margin-bottom: 12px; color: #4E342E; }

/* books */
.book-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; margin-top: 12px; }

```

```
.book-card {
  background: white; border: 1px solid #ddd;
  border-radius: 6px; padding: 14px;
}
.book-card h3 { font-size: 15px; margin-bottom: 4px; }
.book-card .author { font-size: 13px; color: #888; margin-bottom: 6px; }
.book-card .desc { font-size: 13px; color: #555; margin-bottom: 8px; }
.book-card a {
  font-size: 13px; color: #E65100;
  text-decoration: none; font-weight: bold;
}
.book-card a:hover { text-decoration: underline; }

/* forms */
.form-box {
  background: white; border: 1px solid #ddd;
  border-radius: 6px; padding: 20px;
  max-width: 400px; margin-top: 12px;
}
.form-box label { display: block; font-size: 13px; margin-bottom: 4px; color: #555; }
.form-box input[type=text],
.form-box input[type=password] {
  width: 100%; padding: 8px 10px;
  border: 1px solid #ccc; border-radius: 4px;
  font-size: 14px; margin-bottom: 12px;
}
.form-box button {
  padding: 8px 20px; background: #5D4037;
  color: white; border: none; border-radius: 4px;
  font-size: 14px; cursor: pointer;
}
.form-box button:hover { background: #4E342E; }

/* tables */
table {
  width: 100%; border-collapse: collapse;
  margin-top: 10px; font-size: 13px;
}
th, td { padding: 8px 12px; border: 1px solid #ddd; text-align: left; }
th { background: #EFEFE9; }

/* messages */
.msg-ok { color: #2E7D32; font-weight: bold; margin: 10px 0; }
.msg-err { color: #C62828; font-weight: bold; margin: 10px 0; }

/* file content box */
pre.file-content {
  background: #f5f5f5; border: 1px solid #ddd;
  padding: 12px; border-radius: 4px;
  overflow-x: auto; font-size: 13px;
  margin-top: 8px; max-height: 400px;
  overflow-y: auto;
}

/* search */
.search-bar { display: flex; gap: 8px; margin-bottom: 16px; }
.search-bar input[type=text] {
  flex: 1; padding: 8px 10px;
  border: 1px solid #ccc; border-radius: 4px; font-size: 14px;
}
.search-bar button {
  padding: 8px 16px; background: #E65100;
```

```

    color: white; border: none; border-radius: 4px;
    font-size: 14px; cursor: pointer;
}
.search-bar button:hover { background: #BF360C; }

.info-box {
    background: #FFF3E0; border: 1px solid #FFE0B2;
    border-radius: 6px; padding: 12px;
    font-size: 13px; color: #E65100;
    margin-top: 12px;
}

@media (max-width: 600px) { .book-grid { grid-template-columns: 1fr; } }
"""

```

```

def page_layout(title, content):
    return (
        "<!DOCTYPE html><html lang='en'><head>"
        "<meta charset='UTF-8'>"
        "<meta name='viewport' content='width=device-width, initial-scale=1.0'>"
        "<title>" + html_mod.escape(title) + " - BookWorm</title>"
        "<style>" + SITE_CSS + "</style>"
        "</head><body>"
        "<div class='topbar'>"
        "  <span class='logo'>BookWorm Online Library</span>"
        "  <nav>"
        "    <a href='/'>Home</a>"
        "    <a href='/login'>Login</a>"
        "    <a href='/search'>Search</a>"
        "    <a href='/download'>Download</a>"
        "  </nav>"
        "</div>"
        "<div class='container'>"
        + content +
        "</div>"
        "</body></html>"
    )

def get_home_page():
    conn = sqlite3.connect(DEMO_DB)
    cursor = conn.cursor()
    cursor.execute("SELECT id, title, author, filename, description FROM books")
    books = cursor.fetchall()
    conn.close()

    html = "<h2>Welcome to BookWorm</h2>"
    html += "<p>Browse our collection of free programming books.</p>"
    html += "<div class='book-grid'>"
    for book in books:
        book_id, title, author, filename, desc = book
        html += (
            "<div class='book-card'>"
            "<h3>" + html_mod.escape(title) + "</h3>"
            "<div class='author'>by " + html_mod.escape(author) + "</div>"
            "<div class='desc'>" + html_mod.escape(desc) + "</div>"
            "<a href='/download?file=" + html_mod.escape(filename) + "'>Download</a>"
            "</div>"
        )
    html += "</div>"
    return page_layout("Home", html)

```

```

def get_login_form():
    html = (
        "<h2>Login</h2>"
        "<div class='form-box'>"
        "<form method='POST' action='/login'>"
        "  <label>Username</label>"
        "  <input type='text' name='username' placeholder='Enter username'>"
        "  <label>Password</label>"
        "  <input type='password' name='password' placeholder='Enter password'>"
        "  <button type='submit'>Log In</button>"
        "</form>"
        "</div>"
        "<div class='info-box'>"
        "Hint: try logging in with username <b>admin</b> and password <b>s3cr3t!</b>"
        "</div>"
    )
    return page_layout("Login", html)

def handle_login(username, password):
    # ■■■■■ ■-SQLi ■■■■■■■■
    query = (
        "SELECT id, username, password, email, role "
        "FROM users "
        "WHERE username='" + username + "'"
        "AND password='" + password + "'"
    )

    conn = sqlite3.connect(DEMO_DB)
    cursor = conn.cursor()
    rows = []
    error_msg = ""
    try:
        cursor.execute(query)
        rows = cursor.fetchall()
    except Exception as e:
        error_msg = str(e)
    conn.close()

    html = "<h2>Login Result</h2>"

    html += (
        "<p style='font-size:13px;color:#888;'>SQL query that was executed:</p>"
        "<pre class='file-content'>" + html_mod.escape(query) + "</pre>"
    )

    if error_msg:
        html += "<p class='msg-err'>SQL Error: " + html_mod.escape(error_msg) + "</p>"
    elif rows:
        html += (
            "<p class='msg-err'>Returned "
            + str(len(rows)) + " row(s) &mdash; data leaked!</p>"
            "<table><tr>"
            "<th>ID</th><th>Username</th><th>Password</th>"
            "<th>Email</th><th>Role</th></tr>"
        )
        for row in rows:
            html += "<tr>"
            for val in row:
                html += "<td>" + html_mod.escape(str(val)) + "</td>"

```

```

        html += "</tr>"
    html += "</table>"
else:
    html += "<p class='msg-ok'>Login failed &mdash; no matching user.</p>"

html += "<p style='margin-top:16px;'><a href='/login'>Try again</a></p>"
return page_layout("Login Result", html)

def get_search_page(query_text):
    html = (
        "<h2>Search Books</h2>"
        "<form class='search-bar' method='GET' action='/search'>"
        "<input type='text' name='q' placeholder='Search by title or author...' "
        "value='" + html_mod.escape(query_text) + "'>"
        "<button type='submit'>Search</button>"
        "</form>"
    )

    if query_text:
        # ■■■■■ ■-XSS ■■■■■■■
        html += "<p>Results for: <b>" + query_text + "</b></p>"

        conn = sqlite3.connect(DEMO_DB)
        cursor = conn.cursor()
        safe_query = "%" + query_text + "%"
        cursor.execute(
            "SELECT id, title, author, filename, description FROM books "
            "WHERE title LIKE ? OR author LIKE ?",
            (safe_query, safe_query)
        )
        books = cursor.fetchall()
        conn.close()

        if books:
            html += "<div class='book-grid'>"
            for book in books:
                book_id, title, author, filename, desc = book
                html += (
                    "<div class='book-card'>"
                    "<h3>" + html_mod.escape(title) + "</h3>"
                    "<div class='author'>by " + html_mod.escape(author) + "</div>"
                    "<div class='desc'>" + html_mod.escape(desc) + "</div>"
                    "<a href='/download?file=" + html_mod.escape(filename) + "'>Download</a>"
                    "</div>"
                )
            html += "</div>"
        else:
            html += "<p style='color:#999;'>No books matched your search.</p>"
    else:
        html += "<p style='color:#999;'>Enter a search term above.</p>"

    return page_layout("Search", html)

def handle_download(filename):
    if not filename:
        # ■■■■ file -> ■■■■■ ■■■■■■■
        conn = sqlite3.connect(DEMO_DB)
        cursor = conn.cursor()
        cursor.execute("SELECT title, filename FROM books")
        books = cursor.fetchall()

```

```

conn.close()

html = "<h2>Download Books</h2>"
html += "<p>Click a link to view the book content:</p>"
html += "<ul style='margin-top:10px;'>"
for title, fname in books:
    html += (
        "<li style='margin:6px 0;'>"
        "<a href='/download?file=" + html_mod.escape(fname) + "'>"
        + html_mod.escape(title) + "</a>"
        " <span style='color:#999;font-size:12px;'>(" + html_mod.escape(fname) + ")</s
pan>"
        "</li>"
    )
html += "</ul>"
html += (
    "<div class='info-box'>"
    "Hint: try changing the <code>file=</code> parameter in the URL "
    "to <code>../../secrets.txt</code> to attempt path traversal."
    "</div>"
)
return page_layout("Download", html)

# ■■■■■ ■-path traversal ■■■■■
full_path = os.path.join(BOOKS_DIR, filename)
resolved = os.path.abspath(full_path)
allowed_dir = os.path.abspath(BOOKS_DIR)

html = "<h2>File Viewer</h2>"
html += "<p><b>Requested:</b> " + html_mod.escape(filename) + "</p>"
html += "<p><b>Resolved to:</b> " + html_mod.escape(resolved) + "</p>"

if not resolved.startswith(allowed_dir):
    html += (
        "<p class='msg-err'>"
        "PATH TRAVERSAL: this file is OUTSIDE the books directory!</p>"
    )
else:
    html += "<p class='msg-ok'>Path is inside the books directory.</p>"

if os.path.isfile(resolved):
    try:
        with open(resolved, "r", errors="replace") as f:
            content = f.read(4096)
            html += "<p><b>File content:</b></p>"
            html += "<pre class='file-content'>" + html_mod.escape(content) + "</pre>"
    except Exception as e:
        html += "<p class='msg-err'>Error reading file: " + html_mod.escape(str(e)) + "</p>"
    else:
        html += "<p style='color:#999;'>File not found at this path.</p>"

html += "<p style='margin-top:16px;'><a href='/download'>Back to list</a></p>"
return page_layout("File Viewer", html)

def handle_route(request, client_socket):
    path = request.path or "/"

    if path == "/":
        client_socket.sendall(build_response(get_home_page()))

```

```

elif path == "/login":
    if request.method == "POST":
        username = request.form.get("username", [""])[0]
        password = request.form.get("password", [""])[0]
        page = handle_login(username, password)
    else:
        page = get_login_form()
    client_socket.sendall(build_response(page))

elif path == "/search":
    query_text = request.query_params.get("q", [""])[0]
    page = get_search_page(query_text)
    client_socket.sendall(build_response(page))

elif path == "/download":
    filename = request.query_params.get("file", [""])[0]
    page = handle_download(filename)
    client_socket.sendall(build_response(page))

else:
    page = page_layout("Not Found", "<h2>404 - Page Not Found</h2>")
    client_socket.sendall(build_response(page))

# ■■■■ ■■ ■-response ■■■■ ■■■■ ■■■■■■
class SecureSocket:
    def __init__(self):
        self.buffer = b""

    def sendall(self, data):
        self.buffer += data

def handle_client(client_socket):
    try:
        aes_key = crypto_utils.perform_server_handshake(client_socket)

        encrypted_request = crypto_utils.recv_msg(client_socket)
        if not encrypted_request:
            return

        request_data = crypto_utils.aes_decrypt(aes_key, encrypted_request)
        request = HTTPRequest(request_data)

        secure_socket = SecureSocket()
        handle_route(request, secure_socket)

        if secure_socket.buffer:
            encrypted_response = crypto_utils.aes_encrypt(aes_key, secure_socket.buffer)
            crypto_utils.send_msg(client_socket, encrypted_response)

    except Exception as e:
        #print("DEBUG handle_client:", request_data[:100])
        print("Error: " + str(e))
    finally:
        client_socket.close()

def main():
    init_demo_db()

    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

```

```
server.bind(("127.0.0.1", 5000))
server.listen(5)
print("BookWorm server running on port 5000")

while True:
    client_socket, address = server.accept()
    thread = threading.Thread(target=handle_client, args=(client_socket,))
    thread.start()

if __name__ == "__main__":
    main()
```

```
import urllib.parse
import json

class HTTPRequest:
    def __init__(self, raw_data):
        self.raw = raw_data
        self.method = None
        self.path = None
        self.version = None
        self.headers = {}
        self.body = ""
        self.query_params = {}
        self.form = {}
        self.json = None
        self.error = None

        self.parse()

    def parse(self):
        try:
            text = self.raw.decode(errors="ignore")

            if "\r\n\r\n" not in text:
                self.error = "Invalid HTTP Format"
                return

            header_part, _, body_part = text.partition("\r\n\r\n")
            self.body = body_part

            lines = header_part.split("\r\n")
            if not lines:
                self.error = "Empty Request"
                return

            request_line = lines[0]
            parts = request_line.split()
            if len(parts) != 3:
                self.error = "Malformed Request Line"
                return

            self.method, full_path, self.version = parts

            allowed_methods = ["GET", "POST", "PUT", "DELETE", "OPTIONS", "HEAD", "PATCH"]
            if self.method not in allowed_methods:
                self.error = "Invalid Method"
                return

            if self.version not in ["HTTP/1.0", "HTTP/1.1"]:
                self.error = "Invalid HTTP Version"
                return

            # path + query
            parsed_url = urllib.parse.urlparse(full_path)
            self.path = parsed_url.path
            self.query_params = urllib.parse.parse_qs(parsed_url.query)

            # headers
            for line in lines[1:]:
                if ":" in line:
                    key, value = line.split(":", 1)
                    self.headers[key.strip().lower()] = value.strip()
```

```
    if len(self.headers) > 50:
        self.error = "Too Many Headers"
        return

    if self.version == "HTTP/1.1" and "host" not in self.headers:
        self.error = "Missing Host Header"
        return

    # parse body ■■■ Content-Type
    content_type = self.headers.get("content-type", "").lower()
    if self.body:
        self.parse_body(content_type)

    except Exception as e:
        self.error = "Parsing Error"

def parse_body(self, content_type):
    try:
        # form data
        if "application/x-www-form-urlencoded" in content_type:
            self.form = urllib.parse.parse_qs(self.body)
            return

        # json
        if "application/json" in content_type and self.body:
            self.json = json.loads(self.body)
    except Exception:
        self.error = "Body Parsing Error"
```

```

__author__ = 'Yossi'

# from tcp_by_size import send_with_size ,recv_by_size

SIZE_HEADER_FORMAT = "00000000|" # n digits for data size + one delimiter
size_header_size = len(SIZE_HEADER_FORMAT)
TCP_DEBUG = True
LEN_TO_PRINT = 100

def recv_by_size(sock):
    size_header = b''
    data_len = 0
    while len(size_header) < size_header_size:
        _s = sock.recv(size_header_size - len(size_header))
        if _s == b'':
            size_header = b''
            break
        size_header += _s
    data = b''
    if size_header != b'':
        data_len = int(size_header[:size_header_size - 1])
        while len(data) < data_len:
            _d = sock.recv(data_len - len(data))
            if _d == b'':
                data = b''
                break
            data += _d

    if TCP_DEBUG and size_header != b'':
        print ("\nRecv(%s)>>>" % (size_header, ), end='')
        print ("%s"%(data[:min(len(data), LEN_TO_PRINT)], ))
    if data_len != len(data):
        data=b'' # Partial data is like no data !
    return data

def send_with_size(sock, bdata):
    if type(bdata) == str:
        bdata = bdata.encode()
    len_data = len(bdata)
    header_data = str(len(bdata)).zfill(size_header_size - 1) + "|"

    bytea = bytearray(header_data, encoding='utf8') + bdata

    sock.send(bytea)
    if TCP_DEBUG and len_data > 0:
        print ("\nSent(%s)>>>" % (len_data, ), end='')
        print ("%s"%(bytea[:min(len(bytea), LEN_TO_PRINT)], ))

```

```

import socket
import re
import threading
from http_parser import HTTPRequest
import urllib.parse
import time
from collections import deque
import sqlite3
import json
import crypto_utils
import os
import tcp_by_size

DB_NAME = "waf_logs.db"
DRY_RUN = True # True = dry run, False = blocking

def init_db():
    conn = sqlite3.connect(DB_NAME)
    cur = conn.cursor()
    cur.execute("""
        CREATE TABLE IF NOT EXISTS attack_logs (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
            ip TEXT,
            url TEXT,
            reason TEXT
        )
    """)
    conn.commit()
    conn.close()

def log_attack(ip, url, reason):
    try:
        conn = sqlite3.connect(DB_NAME)
        cur = conn.cursor()
        cur.execute("INSERT INTO attack_logs (ip, url, reason) VALUES (?, ?, ?)", (ip, str(url
), reason))
        conn.commit()
        conn.close()
    except Exception as e:
        print(f"[Log Error] {e}")

# RATE_LIMITING
rate_max=20
rate_sec=10

rate_map={} #ip:time_start,count_req
rate_lock = threading.Lock()

#DDOS - ■■■■ ■■■ IP-■■ ■■■■■ ■■■■■■
max_ips_per_sec=5
ATTACK_COOLDOWN_SEC = 30
conn_history = deque() # (timestamp, ip)
conn_lock = threading.Lock()
attack_until = 0

verified_ips = {}
verified_lock = threading.Lock()
CHALLENGE_TIMEOUT = 300 # 5 ■■■■

# IP Blacklisting
BLACKLIST_TIME_SEC = 300 # 5 minutes

```

```

MAX_OFFENSES = 3
blacklist_map = {}
blacklist_lock = threading.Lock()

def record_suspicious_activity(ip):
    now = time.time()
    with blacklist_lock:
        if ip not in blacklist_map:
            blacklist_map[ip] = [0, 0]

        if blacklist_map[ip][1] > now:
            return

        blacklist_map[ip][0] += 1
        if blacklist_map[ip][0] >= MAX_OFFENSES:
            blacklist_map[ip][1] = now + BLACKLIST_TIME_SEC
            print(f"[Blacklist] IP {ip} banned for {BLACKLIST_TIME_SEC}s")

def is_ip_blacklisted(ip):
    now = time.time()
    with blacklist_lock:
        if ip in blacklist_map:
            banned_until = blacklist_map[ip][1]
            if banned_until > now:
                return True
            elif banned_until != 0 and banned_until <= now:
                blacklist_map[ip] = [0, 0]
    return False

SQLI_PATTERNS = [
    r"(?i)\bor\b\s+1\s*=\s*1\b",
    r"(?i)union\s+select",
    r"--",
]
XSS_PATTERNS = [
    r"<\s*script\b",
    r"on\w+\s*=",
    r"javascript\s*:",
    r"<\s*img\b",
]
PATH_TRAVERSAL_PATTERNS = [
    r"\.\./",
    r"\.\.\.",
    r"%2[eE]%2[eE]%2[fF]",
    r"%2[eE]%2[eE]%5[cC]"
]

MAX_HEADER_SIZE = 8192 # 8KB
MAX_BODY_SIZE = 2 * 1024 * 1024

def receive_full_request(client_socket):
    buffer = b""

    try:
        while b"\r\n\r\n" not in buffer:
            chunk = client_socket.recv(1024)
            if not chunk:
                return b""
            buffer += chunk
        if len(buffer) > MAX_HEADER_SIZE:
            return b"HEADER_TOO_LARGE"
    
```

```

headers_part, body_part = buffer.split(b"\r\n\r\n", 1)

headers_text = headers_part.decode(errors="ignore")
content_length = 0

for line in headers_text.split("\r\n"):
    if line.lower().startswith("content-length"):
        try:
            content_length = int(line.split(":")[1].strip())
        except ValueError:
            content_length = 0

if content_length > MAX_BODY_SIZE:
    return b"BODY_TOO_LARGE"

while len(body_part) < content_length:
    chunk = client_socket.recv(1024)
    if not chunk:
        return b""
    body_part += chunk
    if len(body_part) > MAX_BODY_SIZE:
        return b"BODY_TOO_LARGE"

return headers_part + b"\r\n\r\n" + body_part

except (ConnectionResetError, socket.timeout):
    return b""

def send_response(client_socket, body, status="200 OK", content_type="text/html", connection="close"):
    if isinstance(body, str):
        body = body.encode("utf-8")
    header = (
        f"HTTP/1.1 {status}\r\n"
        f"Content-Type: {content_type}\r\n"
        f"Content-Length: {len(body)}\r\n"
        f"Connection: {connection}\r\n"
        "\r\n"
    ).encode("utf-8")
    try:
        client_socket.sendall(header + body)
    except Exception:
        pass

def build_response_str(body, status="200 OK", content_type="text/plain", connection="close"):
    if isinstance(body, str):
        body = body.encode("utf-8")
    header = (
        f"HTTP/1.1 {status}\r\n"
        f"Content-Type: {content_type}\r\n"
        f"Content-Length: {len(body)}\r\n"
        f"Connection: {connection}\r\n"
        "\r\n"
    ).encode("utf-8")
    return header + body

CHALLENGE_HTML = """
<html>
<body>

```

```
<h2>Security Check</h2>
<form action="/waf_challenge" method="GET">
    Solve: 3 + 7 = ?
    <input name="ans">
    <button type="submit">Submit</button>
</form>
</body>
</html>
"""
```

```
def send_challenge_page(client_socket):
    send_response(client_socket, CHALLENGE_HTML)

def detect_sql_injection(value):
    for pattern in SQLI_PATTERNS:
        if re.search(pattern, value):
            return True
    return False

def detect_xss(value):
    if not isinstance(value, str):
        return False
    for pattern in XSS_PATTERNS:
        if re.search(pattern, value, re.IGNORECASE):
            return True
    return False

def detect_path_traversal(value):
    if not isinstance(value, str):
        return False
    for pattern in PATH_TRAVERSAL_PATTERNS:
        if re.search(pattern, value):
            return True
    return False

def is_request_suspicious(req):
    candidates = []

    if req.path:
        candidates.append(req.path)
    if req.body:
        candidates.append(req.body)

    for values in req.query_params.values():
        candidates.extend(values)

    for values in req.form.values():
        candidates.extend(values)

    for value in candidates:
        if not isinstance(value, str):
            continue

        decoded_value = urllib.parse.unquote_plus(value)
        double_decoded = urllib.parse.unquote_plus(decoded_value)
        normalized_value = double_decoded.lower()

        if detect_sql_injection(normalized_value) or detect_sql_injection(value):
```

```

        return True
    if detect_xss(normalized_value) or detect_xss(value):
        return True
    if detect_path_traversal(normalized_value) or detect_path_traversal(value):
        return True

    return False

def detect_rate_limiting(ip):
    now = time.time()

    with rate_lock:
        if ip not in rate_map:
            rate_map[ip] = [now, 1]
            return False

        win_start, count = rate_map[ip]

        # ■■■ ■■■■■ ■■■■ ■■ ■■■■■■
        if now - win_start >= rate_sec:
            rate_map[ip] = [now, 1]
            return False

        # ■■■■■ ■■■■ ■■■■■
        if count >= rate_max:
            return True

        # ■■■■ ■■■ ■■■■
        rate_map[ip][1] = count + 1
        return False

def is_under_ddos_attack(now):
    return now < attack_until

def update_attack_state(now, ip):
    global attack_until

    with conn_lock:
        conn_history.append((now, ip))

        # ■■■■■■ ■■■■■■■ ■■■■■ ■■■■ ■■■■■■
        while conn_history and now - conn_history[0][0] > 1:
            conn_history.popleft()

        # ■■■■■■ ■■■ IP-■■ ■■■■■ ■■■■■■ ■■■■■■ ■■■■■■
        unique_ips = set()
        for t, addr in conn_history:
            unique_ips.add(addr)

        # DDoS ■■ ■■ ■■ ■■■■ IP-■■ ■■■■■, ■■ ■■ IP ■■■ ■■■■■ ■■■■
        if len(unique_ips) > max_ips_per_sec:
            if now >= attack_until:
                print(f"[DDoS] Under Attack! {len(unique_ips)} unique IPs/sec")
                attack_until = now + ATTACK_COOLDOWN_SEC

def is_verified(ip):
    now = time.time()
    with verified_lock:
        if ip in verified_ips:
            if verified_ips[ip] > now:

```

```

        return True
    else:
        del verified_ips[ip]
    return False

def set_verified(ip):
    with verified_lock:
        expiry_timestamp = time.time() + CHALLENGE_TIMEOUT
        verified_ips[ip] = expiry_timestamp
        print(f"[Challenge] Verified IP: {ip} for {CHALLENGE_TIMEOUT} sec")

def send_error_response(client_socket, status, body):
    send_response(client_socket, body, status=status, content_type="text/plain")

def get_events(since_id):
    try:
        conn = sqlite3.connect(DB_NAME)
        conn.row_factory = sqlite3.Row
        cursor = conn.cursor()
        cursor.execute(
            "SELECT id, timestamp, ip, url, reason "
            "FROM attack_logs WHERE id > ? "
            "ORDER BY id DESC LIMIT 100",
            (since_id,)
        )
        rows = [dict(row) for row in cursor.fetchall()]
        conn.close()
        return rows
    except Exception:
        return []

def send_html_response(client_socket, html, connection="close"):
    send_response(client_socket, html, content_type="text/html; charset=utf-8", connection=connection)

def send_json_response(client_socket, data, connection="close"):
    send_response(client_socket, json.dumps(data), content_type="application/json", connection=connection)

DASHBOARD_HTML_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "waf_proxy_dashboard.html")

def get_events_page_html():
    with open(DASHBOARD_HTML_FILE, "r", encoding="utf-8") as f:
        return f.read()

def handle_ddos_challenge(req, ip, client_socket, detected_attacks):
    if is_under_ddos_attack(time.time()) and not is_verified(ip):
        log_attack(ip, getattr(req, "path", "-"), "DDoS Challenge Triggered")
        detected_attacks.append("DDoS Challenge Triggered")

    if not DRY_RUN:
        if req.path and req.path.startswith("/waf_challenge"):
            answer = req.query_params.get("ans", [""])[0]

            if answer == "10":
                set_verified(ip)
                send_response(client_socket, "<h2>Verified. You can return to the site.</h2>")

```

2&gt;)

```

        return True
    else:
        send_challenge_page(client_socket)
        return True
    else:
        send_challenge_page(client_socket)
        return True

    return False

def run_waf_checks(req, ip, client_socket, detected_attacks):
    now = time.time()

    # DDoS
    if handle_ddos_challenge(req, ip, client_socket, detected_attacks):
        return "break"

    # Rate limit
    if detect_rate_limiting(ip):
        log_attack(ip, getattr(req, "path", "-"), "Rate Limiting")
        detected_attacks.append("Rate Limiting")
        return "continue"

    # Suspicious payload
    if is_request_suspicious(req):
        record_suspicious_activity(ip)
        log_attack(ip, getattr(req, "path", "-"), "Suspicious Payload")
        detected_attacks.append("Suspicious Payload")

        if not DRY_RUN:
            return "break"

    return None

def apply_waf_policy(detected_attacks, client_socket):
    # block or pass through
    if detected_attacks and not DRY_RUN:
        if "IP Blacklisted" in detected_attacks or "Suspicious Payload" in detected_attacks:
            send_error_response(client_socket, "403 Forbidden", "Blocked by WAF")
            return "break"

        elif "Rate Limiting" in detected_attacks:
            send_response(
                client_socket,
                "Too Many Requests",
                status="429 Too Many Requests",
                content_type="text/plain",
                connection="keep-alive"
            )
            return "continue"

        elif "DDoS Challenge Triggered" not in detected_attacks:
            send_error_response(client_socket, "400 Bad Request", "Bad Request")
            return "break"

    return None

def add_waf_debug_header(response, detected_attacks):
    if not DRY_RUN or not detected_attacks:
        return response

```

```

attacks_str = ", ".join(detected_attacks).encode("utf-8")
header = b"X-WAF-Detected: " + attacks_str + b"\r\n"

if b"\r\n" in response:
    parts = response.split(b"\r\n", 1)
    return parts[0] + b"\r\n" + header + parts[1]

return response + header

def handle_dashboard(request_data, client_socket):
    first_line = request_data.split(b"\r\n")[0].decode()
    url_part = first_line.split(" ")[1]

    if url_part.startswith("/waf_events/api"):
        query_string = url_part.split("?", 1)[1] if "?" in url_part else ""
        params = urllib.parse.parse_qs(query_string)

        since_id = 0
        try:
            since_id = int(params.get("since_id", ["0"])[0])
        except (ValueError, IndexError):
            pass

        events = get_events(since_id)
        send_json_response(client_socket, events, connection="keep-alive")

    else:
        html = get_events_page_html()
        send_html_response(client_socket, html, connection="keep-alive")

    return True

def handle_waf_control(request_data, client_socket):
    global DRY_RUN

    # ■■■■■ ■URL
    first_line = request_data.split(b"\r\n")[0].decode()
    url_part = first_line.split(" ")[1]
    query_string = url_part.split("?", 1)[1] if "?" in url_part else ""
    params = urllib.parse.parse_qs(query_string)
    action = params.get("action", ["toggle_mode"])[0]

    if action == "toggle_mode":
        DRY_RUN = not DRY_RUN
        result = {"mode": "dry_run" if DRY_RUN else "blocking"}

    elif action == "get_status":
        now = time.time()
        blocked_ips = []
        with blacklist_lock:
            for blocked_ip, info in blacklist_map.items():
                banned_until = info[1]
                if banned_until > now:
                    blocked_ips.append({"ip": blocked_ip, "expires_in": int(banned_until - now)})

    result = {
        "mode": "dry_run" if DRY_RUN else "blocking",
        "blacklist": blocked_ips
    }

    elif action == "add_blacklist":
        target_ip = params.get("ip", [""])[0]

```

```

    if target_ip:
        with blacklist_lock:
            blacklist_map[target_ip] = [MAX_OFFENSES, time.time() + 999999]
            result = {"ok": True, "ip": target_ip}
        else:
            result = {"ok": False, "error": "No IP provided"}

    elif action == "remove_blacklist":
        target_ip = params.get("ip", [""])[0]
        if target_ip:
            with blacklist_lock:
                if target_ip in blacklist_map:
                    del blacklist_map[target_ip]
                result = {"ok": True, "ip": target_ip}
            else:
                result = {"ok": False, "error": "No IP provided"}

        else:
            result = {"error": "Unknown action"}

    send_json_response(client_socket, result, connection="keep-alive")
    return True

def handle_client(client_socket, client_addr):
    try:
        ip = client_addr[0]
        client_socket.settimeout(30) # timeout ████████ keep-alive

        while True:
            detected_attacks = []
            if is_ip_blacklisted(ip):
                log_attack(ip, "-", "IP Blacklisted")
                detected_attacks.append("IP Blacklisted")
            request_data = receive_full_request(client_socket)
            if not request_data:
                return

            # waf control api
            if request_data.startswith(b"GET /waf_control?secret=supersecretwafkey"):
                handle_waf_control(request_data, client_socket)
                continue

            # dashboard page
            if request_data.startswith(b"GET /waf_events"):
                handle_dashboard(request_data, client_socket)

            if request_data == b"HEADER_TOO_LARGE":
                log_attack(ip, "-", "Header Too Large")
                send_error_response(client_socket, "431 Request Header Fields Too Large", "Header Too Large")
                break

            if request_data == b"BODY_TOO_LARGE":
                log_attack(ip, "-", "Body Too Large")
                send_error_response(client_socket, "413 Payload Too Large", "Body Too Large")
                break

            req = HTTPRequest(request_data)
            if req.error:
                path = getattr(req, "path", "-")
                if not path:
                    path = "-"
                log_attack(ip, path, f"Bad Request: {req.error}")

```

```

        detected_attacks.append(f"Bad Request: {req.error}")

    # DDoS / rate limit / security checks
    action = run_waf_checks(req, ip, client_socket, detected_attacks)
    if action == "break":
        break
    elif action == "continue":
        continue

    # block or pass through
    action = apply_waf_policy(detected_attacks, client_socket)
    if action == "break":
        break
    elif action == "continue":
        continue

    # forward to backend
    backend_response = forward_to_backend(request_data)
    # ■■■ ■■ ■■■■■ ■■■ ■■■■■ ■■■■■ header
    backend_response = add_waf_debug_header(backend_response, detected_attacks)
    client_socket.sendall(backend_response)

    # check if client wants to close
    conn = req.headers.get("connection", "")
    if conn.lower() == "close":
        break

finally:
    client_socket.close()

def forward_to_backend(raw_data):
    backend = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    try:
        backend.connect(("127.0.0.1", 5000))

        # TLS handshake ■■ ■backend
        aes_key = crypto_utils.perform_client_handshake(backend)

        # ■■■■■ ■■■■■
        encrypted_req = crypto_utils.aes_encrypt(aes_key, raw_data)
        crypto_utils.send_msg(backend, encrypted_req)

        # ■■■■ ■■■■■
        encrypted_resp = crypto_utils.recv_msg(backend)
        if encrypted_resp:
            return crypto_utils.aes_decrypt(aes_key, encrypted_resp)
        return b""

    except Exception as e:
        #print("DEBUG forward:", raw_data[:80])
        print(f"Backend Connection/TLS Error: {e}")
        return build_response_str("Backend SSL/TLS Handshake Error or Backend Down", status="502 Bad Gateway")

finally:
    try:
        backend.close()
    except Exception:
        pass

def main():

```

```
init_db()
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind(("127.0.0.1", 8000))
server.listen(100)

print("WAF running on 8000")

while True:
    client_socket, client_addr = server.accept()
    now = time.time()

    # ddos tracking
    update_attack_state(now, client_addr[0])

    thread = threading.Thread(target=handle_client, args=(client_socket, client_addr))
    thread.start()

if __name__ == "__main__":
    main()
```